# Integrating Vision-Language-Action Models into SAILOR for Robust Robotic Imitation Learning

by

Pierre Ishak

Supervisor: Professor Nicholas Rhinehart
April 7th, 2026

**B.A.Sc. Thesis**

Division of Engineering Science
UNIVERSITY OF TORONTO

**Abstract**

This thesis investigates whether $\pi_0$-Droid, a pretrained Vision–Language–Action policy, can serve as a base policy within SAILOR, a latent-search recovery framework for robotic imitation learning. The goal is to combine broad pretrained visuomotor priors with search-based correction under distribution shift without collecting additional task-specific data. The work proceeded in four stages: reproducing the published SAILOR RoboMimic baseline, enabling $\pi_0$-Droid inference in RoboMimic through observation and control-interface alignment, task-aligning $\pi_0$-Droid with JAX LoRA fine-tuning, and integrating the fine-tuned policy into the full SAILOR loop. Fine-tuning substantially improved standalone $\pi_0$-Droid performance on Lift and Can: Lift success increased from 0% to 48%, 66%, and 82% for 5, 10, and 15 demonstrations, while Can improved to 14%, 16%, and 20%. Square remained at 0% success, but mean reward improved from about -199.5 to -166.4, indicating meaningful task progress without reliable completion. After integration, SAILOR still improved performance over the stronger VLA base, especially on Lift and more modestly on Can, although the reproduced diffusion-policy SAILOR baseline remained stronger overall under the fixed benchmark conditions used here. These results show that $\pi_0$-Droid and SAILOR are meaningfully compatible and that integrating a pretrained VLA into SAILOR is technically feasible, but that the remaining challenges are optimization, stability, and tuning rather than interface alignment.

**Acknowledgements**

I would first like to thank God, my Lord and Saviour Jesus Christ, for guiding and sustaining me throughout this thesis, my Engineering Science degree, and this work as a whole. I also thank St. Mari Mina the Wonderworker and St. Pope Kyrillos VI for giving me courage to continue and complete this thesis despite how daunting it felt when I began. I am deeply grateful to my family, including my parents, grandparents, and friends, who supported me through the most stressful parts of this work with their encouragement, prayers, patience, kindness, and smiles. I especially thank my very best friend for reading through my full interim report, helping me edit it and understand it more clearly, and supporting me emotionally throughout the process. I am sincerely grateful to Dr. Rollen S. D'Souza Peng for his guidance throughout this thesis and for meeting with me biweekly on Monday nights for over six months. Those meetings meant a great deal to me, and I deeply appreciate the time, care, and generosity he gave to something that mattered deeply to my life and education. I would also like to thank Sebastian Aegidius for his guidance in the later half of the thesis, for meeting with me weekly, for helping me focus step by step, and for showing genuine empathy and care whenever I was stuck. I thank Professor Nicholas Rhinehart for the opportunity to work with the LEAF Lab, for helping define the starting point of this project through the relevant papers, and for providing access to Compute Canada resources. I also appreciate his role in connecting me with additional support during the project. I am also grateful to my classmates, and especially to Professor Romkey at the beginning of the project, for listening to me talk through this thesis and its challenges. Finally, I would like to thank Mohamed Nabil for his kindness at the start of the project, for reassuring me that struggling with Compute Canada, Apptainer, and environment setup was normal, and for spending time helping me debug during the early stages before I eventually resolved those issues through many more months of work.

**Table of Contents**

1 Introduction ..... 1

1.1 Background and Context ..... 1

1.2 Motivation and Research Gap ..... 2

1.3 Thesis Goal ..... 3

1.4 Significance ..... 4

2 Literature Review ..... 4

2.1 Conceptual and Theoretical Foundations ..... 5

2.1.1 Imitation Learning and Behavioural Cloning ..... 5

2.1.2 The Problem of Distribution Shift ..... 6

2.1.3 Model-Based Reasoning and World Models ..... 9

2.1.4 Dreamer and Latent Imagination-Based Planning ..... 11

2.1.5 Learning to Search and Planning in Latent Spaces ..... 12

2.1.6 The SAILOR Framework ..... 14

2.1.7 Limitations of SAILOR and Motivation for Stronger Base Policies ..... 21

2.1.8 Pretrained Base Policies and Foundation Models in Robotics ..... 21

2.2 Comparative Review ..... 23

2.2.1 Learning-to-Search (L2S) Methods ..... 23

2.2.2 Reinforcement Learning on Top of Imitation Learning (RL-on-IL) ..... 24

2.2.3 Distillation of Planning Results into Policies ..... 26

2.2.4 Vision–Language–Action (VLA) Models as Strong Imitation Priors ..... 27

2.2.5 Summary and Positioning of This Thesis ..... 28

2.3 Literature Review Summary ..... 29

3 Reproducing the SAILOR Baseline ..... 29

3.1 Compute Infrastructure and Apptainer-Based Software Environment ..... 29

3.2 RoboMimic Dataset Preparation and Preprocessing ..... 30

3.3 Published SAILOR Baseline and Benchmark Scope ..... 31

3.4 Baseline Training Pipeline ..... 32

3.5 Evaluation Protocol and Policy Variants ..... 33

3.6 Baseline Results ..... 35

3.7 Interpretation and Takeaways ..... 36

3.8 Chapter Summary ..... 36

4 Enabling $\pi_0$-Droid Inference in RoboMimic ..... 36
4.1 Milestone Objective ..... 36
4.2 Rebuilding the Runtime on Compute Canada ..... 38
4.3 Standalone $\pi_0$-Droid Validation and Scope of the Model Choice ..... 39
4.4 Constructing the Observation Bridge ..... 40
4.5 Camera Settings Available in Simulation ..... 40
4.6 Switching the Simulator to Joint-Velocity Control ..... 41
4.7 Gripper Observation and Execution Mapping ..... 42
4.7.1 Gripper Observation Normalization ..... 42
4.7.2 Gripper Execution Mapping ..... 42
4.8 Prompts, Chunk Execution, and Evaluation Conditions ..... 43
4.9 Zero-Shot Results ..... 44
4.10 Chapter Summary ..... 44
5 Task-Aligning $\pi_0$-Droid through JAX LoRA Fine-Tuning ..... 45
5.1 Milestone Objective ..... 45
5.2 Choosing JAX LoRA Instead of PyTorch Full Fine-Tuning ..... 45
5.3 Building a RoboMimic HDF5 Fine-Tuning Pipeline for OpenPI ..... 46
5.4 JAX LoRA Training Implementation ..... 47
5.5 Training Setup and Benchmark Consistency ..... 48
5.6 Main Fine-Tuning Results at 24,000 Steps ..... 49
5.7 Extended Fine-Tuning to 48,000 Steps ..... 51
5.8 Interpreting Square: Progress Without Completion ..... 53
5.9 Interpretation and Takeaways ..... 53
5.10 Chapter Summary ..... 54
6 Integrating $\pi_0$-Droid as the SAILOR Base Policy ..... 55
6.1 Milestone Objective ..... 55
6.2 Integration Strategy ..... 55
6.3 Code-Level Integration ..... 56
6.4 Training Setup, Compute Cost, and Comparison Views ..... 57
6.5 Best-Round Integrated Results ..... 58
6.6 Latest-Round Results and Training Stability ..... 60
6.7 Matched-Round Comparison Against the Diffusion-Policy Baseline ..... 62

6.8 Best-Available Comparison Against DP + SAILOR ..... 64
6.9 Interpretation and Takeaways ..... 65
6.10 Chapter Summary ..... 66
7 Discussion ..... 67
7.1 Answering the Research Question ..... 67
7.2 What the Results Actually Changed ..... 68
7.3 Why the Current Gap Still Exists ..... 68
7.4 What Should Be Improved Next ..... 69
7.5 Broader Meaning ..... 70
7.6 Chapter Summary ..... 71
8 Conclusion ..... 71
9 References ..... 73

**List of Figures**

Figure 2.1: Algorithm 1 SAILOR (Inference) [4] ..... 18
Figure 2.2: Algorithm 2 SAILOR (Training) [4] ..... 19
Figure 3.1: Original SAILOR Paper Results Comparing Diffusion Policy (DP) and SAILOR on Three RoboMimic Tasks [4]. ..... 32
Figure 3.2: Representative Evaluation Frame from the Square Task Illustrating Success Indicators Across Parallel Rollouts. ..... 34
Figure 3.3: Final Success Rates at the End of Training for the Diffusion-Policy Baseline, Distilled Policy, and Full SAILOR Controller Across All Three Tasks. ..... 35
Figure 4.1: Milestone-2 interface alignment for enabling $\pi_0$-Droid inference in RoboMimic. ..... 37
Figure 4.2: Representative final frame from zero-shot $\pi_0$-Droid evaluation on the Lift task before fine-tuning. ..... 44
Figure 5.1: Success rate for base $\pi_0$-Droid and JAX LoRA fine-tuned $\pi_0$-Droid after 24,000 fine-tuning steps on Lift, Can, and Square. ..... 49
Figure 5.2: Mean reward for base $\pi_0$-Droid and JAX LoRA fine-tuned $\pi_0$-Droid after 24,000 fine-tuning steps. Less negative is better. ..... 50
Figure 5.3: Success rate after extending fine-tuning to 48.000 steps. ..... 51
Figure 5.4: Mean reward after extending fine-tuning to 48,000 steps. ..... 52

Figure 5.5: Representative Square rollout after extended fine-tuning, showing near completion without task success. ..... 53
Figure 6.1: Best-round success for integrated $\pi_0$-Droid + SAILOR runs, compared against the integrated initialization point and the best distilled $\pi_0$-Droid checkpoint within the selected run. ..... 58
Figure 6.2: Best-round mean reward for integrated $\pi_0$-Droid + SAILOR runs. Less negative reward indicates better task progress. ..... 59
Figure 6.3: Latest-round success for integrated $\pi_0$-Droid + SAILOR runs, compared against the integrated initialization point and the latest distilled and online checkpoints available in the selected run. ..... 60
Figure 6.4: Latest-round mean reward for integrated $\pi_0$-Droid + SAILOR runs. Less negative reward is better. ..... 61
Figure 6.5: Matched-round success comparison between DP + SAILOR and $\pi_0$-Droid LoRA + SAILOR. The diffusion-policy baseline is evaluated at the same round index as the best $\pi_0$-Droid + SAILOR round. ..... 62
Figure 6.6: Matched-round mean reward comparison between DP + SAILOR and $\pi_0$-Droid LoRA + SAILOR. Less negative reward is better. ..... 63
Figure 6.7: Best-available success comparison between the reproduced DP + SAILOR baseline and the integrated $\pi_0$-Droid LoRA + SAILOR system. ..... 64
Figure 6.8: Best-available mean reward comparison between the reproduced DP + SAILOR baseline and the integrated $\pi_0$-Droid LoRA + SAILOR system. Less negative reward is better. ..... 65

# 1 Introduction

## 1.1 Background and Context

Robotic imitation learning enables robots to learn manipulation skills by imitating expert demonstrations rather than relying on explicit task programming. In this approach, a policy is trained to reproduce expert behaviour by mapping visual observations and robot states to actions. The most common method is Behavioural Cloning (BC), where a model learns to predict the expert's next action from the current observation. Although BC is simple and effective, it suffers from a limitation known as distribution shift. When the robot encounters states not seen during demonstrations, it cannot recover, leading to compounding errors that degrade performance, especially in long or contact-rich tasks.

Recent progress in robot learning has been driven by two main directions: large-scale datasets and foundation models for generalist robot control. The DROID (A Large-Scale In-the-Wild Robot Manipulation Dataset) dataset introduced over 76,000 real-world trajectories across 564 scenes and 86 manipulation tasks collected on multiple robot platforms. Its scale and diversity showed that generalization in robot learning improves not only with dataset size but also with exposure to varied environments and embodiments [1].

At the same time, foundation models for robotics have extended the success of vision-language models into control. The $\pi_0$ model [2] is a leading example. Trained on Internet-scale multimodal data and thousands of robot demonstrations, $\pi_0$ learns a Vision-Language-Action (VLA) policy that connects natural-language goals, visual inputs, and robot actions. This allows it to perform zero-shot manipulation across many tasks by leveraging shared semantic and visuomotor representations.

$\pi_0$-Droid is obtained by fine-tuning $\pi_0$ on the DROID dataset and is aligned with the DROID hardware stack, which uses a Franka Panda arm with a gripper in a tabletop manipulation setting. Because the RoboMimic tasks used in this thesis are also built around Franka Panda manipulation, $\pi_0$-Droid is a practical candidate for integration into the RoboMimic benchmark and the SAILOR evaluation pipeline [1], [3].

While $\pi_0$ is trained using supervised imitation learning similar to behavioural cloning, it is not a simple feedforward policy. The model uses conditional flow matching to predict distributions of future actions rather than single-step commands, which helps it handle diverse data more robustly

than standard BC. However, $\pi_0$ still does not perform explicit planning or test-time search, and therefore can still suffer from compounding errors under significant distribution shift.

### 1.2 Motivation and Research Gap

The SAILOR framework (Searching Across Imagined Latents Online for Recovery) [4] directly targets the recovery problem in imitation learning. It augments a base imitation policy with learned auxiliary models for prediction and evaluation. During testing, SAILOR performs search in its learned latent space, simulating multiple possible action sequences and selecting the most promising one when the base policy begins to fail. It then executes the first step of that corrected plan before re-planning, similar to model predictive control.

However, SAILOR's effectiveness depends strongly on the quality of its base policy. In its original work, the base policy was a per-task diffusion model trained on limited demonstrations. While effective within its distribution, such a base lacks the broad visuomotor priors and semantic grounding that a pretrained model like $\pi_0$-Droid offers.

Meanwhile, $\pi_0$-Droid generalizes across a wide range of tasks but cannot perform test-time planning or recovery once an error occurs. Each method therefore addresses a different aspect of robustness: $\pi_0$-Droid offers broad generalization, while SAILOR provides recovery through search and adaptation.

This creates a clear research gap:

Can a pretrained Vision-Language-Action model such as $\pi_0$-Droid serve as an effective base policy within SAILOR, combining large-scale pretrained visuomotor priors with SAILOR's test-time recovery and online improvement mechanisms?

This question unites two key directions in robot learning:

1. scalable, foundation-model generalization, and
2. online, search-based recovery for error correction.

To date, no prior work has evaluated how a foundation-model policy like $\pi_0$ performs when integrated into a latent-search framework such as SAILOR.

### 1.3 Thesis Goal

The goal of this thesis is to evaluate whether the pretrained $\pi_0$-Droid model can function as an effective base policy within SAILOR on RoboMimic benchmark tasks without additional data collection. Closed-loop performance is assessed primarily using two simulator-provided metrics: task success and mean reward. The research aims to determine whether broader pretrained visuomotor priors can provide useful nominal action proposals for SAILOR and whether latent-search-based recovery remains beneficial on top of that pretrained prior.

The main objectives are:

1. Reproduce the published SAILOR baseline and validate its reported results on RoboMimic tasks such as Lift, Can, and Square [3], [4]. This establishes a trusted reference point, validates the recovery framework, and provides a clear understanding of the codebase and evaluation pipeline before modifying the system.

2. Establish $\pi_0$ inference in the RoboMimic simulator by aligning the observation pipeline, action contract, and controller interface with the requirements of $\pi_0$ and SAILOR.

3. Fine-tune $\pi_0$ on RoboMimic-style tasks using JAX LoRA so that the pretrained base policy is adapted to the same task family as the original SAILOR diffusion-policy baseline under practical one-GPU memory constraints.

4. Integrate $\pi_0$ into the SAILOR framework by replacing the original diffusion-policy base while preserving SAILOR's learned world model, reward model, and latent-search recovery mechanism.

The remainder of this thesis is organized as follows. Chapter 2 reviews the relevant literature on behavioural cloning, distribution shift, world models, SAILOR, and vision–language–action models. Chapters 3 through 6 then present the work through four stages of development and evaluation: reproducing the SAILOR baseline, enabling $\pi_0$-Droid inference in RoboMimic, task-aligning $\pi_0$-Droid through JAX LoRA fine-tuning, and integrating $\pi_0$-Droid into the full SAILOR framework. Chapter 7 discusses the results across these stages in relation to the research question, and Chapter 8 concludes the thesis.

### 1.4 Significance

The significance of this thesis is not simply that it combines two strong methods. Rather, it tests whether broad pretrained visuomotor priors and latent-search-based recovery can be brought into a single control system in a technically consistent and experimentally meaningful way.

Pretrained Vision–Language–Action models such as $\pi_0$-Droid provide broad visuomotor priors learned from large and diverse robot data, which may yield stronger nominal action proposals than task-specific policies trained only on smaller demonstration sets [1], [2]. SAILOR addresses a different limitation: when a base policy drifts off the demonstration distribution, it performs online search in a learned world model to find corrective residual actions [4]. If $\pi_0$-Droid provides a useful starting policy, then SAILOR's search may be able to focus on smaller or more targeted corrections, potentially improving robustness and making better use of limited task-specific demonstrations.

Such results would have several important implications for robot learning:

- Scalability: reducing reliance on collecting large amounts of additional task-specific demonstration data.
- Robustness: combining stronger pretrained priors with online search-based correction under distribution shift.
- Generalization: extending latent-search recovery frameworks beyond narrow task-specific base policies.
- Practicality: supporting more reliable manipulation by combining generalist priors with test-time corrective planning.

Overall, this thesis investigates whether large pretrained visuomotor priors and latent-search-based recovery can address complementary failure modes in robotic imitation learning.

## 2 Literature Review

This literature review provides the theoretical and empirical background necessary to situate the research within the broader field of robot learning. It outlines how prior work in imitation learning, model-based reasoning, and foundation models has contributed to developing robust and generalizable control policies.

The review is divided into two main parts. The first part, Conceptual and Theoretical Foundations, introduces fundamental concepts that underpin this work. It begins with imitation learning and behavioural cloning, describes their limitations due to distribution shift, and then examines model-based reasoning and world models as predictive approaches for improving robustness. This section concludes with an overview of the SAILOR framework, which achieves recovery through latent-space search and evaluation, and introduces foundation models such as $\pi_0$ that provide large-scale visuomotor priors but do not perform explicit search or evaluation-based reasoning during test time.

The second part, Comparative Review, surveys complementary methods that refine or enhance imitation-learned policies. These include learning-to-search algorithms, reinforcement learning fine-tuning, policy distillation techniques, and recent Vision-Language-Action models that extend imitation learning through large-scale multimodal pretraining.

## 2.1 Conceptual and Theoretical Foundations

### 2.1.1 Imitation Learning and Behavioural Cloning

Imitation Learning (IL) provides a framework for training autonomous agents to perform tasks by learning directly from expert demonstrations rather than through hand-crafted reward functions or manually designed controllers. In this setting, a policy is trained to reproduce expert behaviour by mapping observed states or sensory inputs to corresponding actions. This approach is particularly valuable in robotic manipulation, where it is often easier to record human or teleoperated demonstrations than to engineer effective reward functions or rely on trial-and-error exploration.

Behavioural Cloning (BC) is the most straightforward and widely used form of imitation learning. It formulates policy learning as a supervised learning problem: given a dataset of expert state–action pairs, the policy is optimized to predict the expert's action for each observed state. In practice, this often involves minimizing a regression or classification loss so that the learned policy outputs actions closely matching those taken by the expert. When applied to robotics, BC can use either visual inputs (such as camera images) or proprioceptive data (like joint angles and forces), enabling robots to imitate complex manipulation behaviours.

Frameworks such as RoboMimic have standardized this approach by releasing large-scale datasets and consistent evaluation benchmarks, allowing researchers to train and compare imitation-learning methods on common robotic arms and tasks such as grasping, lifting, and stacking [3].

Despite its simplicity and success as a foundational method for visuomotor learning, BC faces inherent challenges in sequential decision-making and dynamic environments. The most significant of these is distribution shift, which occurs when the learned policy encounters states at test time that were not represented in the expert demonstrations.

To mitigate this problem, interactive imitation learning techniques have been developed. The most influential among them is Dataset Aggregation (DAgger), introduced by Ross et al. [5]. DAgger iteratively refines the policy by executing it in the environment, collecting new states visited by the learner, and querying the expert for the correct actions at those states. Although DAgger substantially reduces distribution shift, it requires repeated expert supervision, which limits its scalability in robotic systems where human or simulated expert feedback can be costly or unsafe.

Recent advances such as Diffusion Policy (DP) [6] extend behavioural cloning by introducing generative action models capable of producing diverse, temporally consistent motion sequences. Instead of directly predicting a single next action, a diffusion-based policy learns to generate an entire sequence of plausible future actions by progressively refining samples of random noise into structured trajectories that match the expert's demonstrations. This process models the probability distribution of expert behaviour rather than a single deterministic mapping.

Diffusion-based action generation models a conditional distribution over future action chunks rather than a single one-step command, while also producing temporally coherent continuous controls [6]. This makes diffusion policies a strong behavioural-cloning baseline for robotic manipulation. However, the ability to model multimodal action distributions does not by itself imply that full trajectories are multimodal in every task setting, and diffusion policies still do not provide an explicit mechanism for online recovery once execution departs from the demonstration distribution.

While DP improves imitation quality and robustness within the demonstration distribution, it still inherits BC's core limitation: reliance on static, offline expert data. The model cannot reason about or correct errors during execution because it lacks mechanisms for online planning or recovery.

### 2.1.2 The Problem of Distribution Shift

As noted in the previous section, BC is limited by distribution shift, a fundamental issue in sequential decision-making. Distribution shift occurs when the state distribution encountered by a

learned policy during execution diverges from the state distribution present in the expert demonstrations used for training.

In standard supervised learning, models are trained under the assumption that all samples are independent and identically distributed (i.i.d.), meaning that each example is drawn independently from the same underlying data distribution. This assumption allows a model to generalize effectively from its training data to unseen examples drawn from the same distribution. In imitation learning, however, this assumption no longer holds because the learner's actions directly determine which states it will visit. A single incorrect action changes the next observation, pushing the system into states that may not have been seen during training. Over time, these small deviations accumulate, resulting in compounding errors that cause the robot to drift further from expert-like behaviour [7].

Recent theoretical work, such as Stable-BC by Mehta et al. [7], formalizes this problem and proposes strategies to mitigate it. Stable-BC introduces stability constraints during training that limit how much the policy's predicted actions can deviate from expert behaviour. By constraining this divergence, the method reduces the rate at which covariate shift grows across time steps, improving stability and reducing cascading error effects. However, since Stable-BC is still trained on offline demonstrations, it cannot completely prevent distribution shift once the policy is deployed and begins influencing its own input distribution.

Empirical studies have reinforced these theoretical findings. Karkus et al. [8] characterize distribution shift as a difference between open-loop and closed-loop imitation learning. In open-loop imitation, the policy is trained only on static expert data without feedback from its own actions, which makes it fragile under real execution. Closed-loop learning, on the other hand, exposes the agent to its own induced trajectories, allowing it to receive corrective supervision or self-generated rollouts. Their analysis shows that closed-loop approaches are significantly more robust, particularly in domains such as autonomous driving where small compounding errors can lead to critical failures.

These results show that purely reactive imitation policies are vulnerable once execution moves off the expert-induced distribution. This motivates methods that can evaluate the consequences of candidate actions after such drift occurs. One important family of approaches does so by learning predictive models of dynamics and using them for planning or search. Foundational work by

Kaelbling, Littman, and Cassandra [9] on partially observable Markov decision processes (POMDPs) formalized the idea that effective decision-making under uncertainty requires an internal belief model of the environment. Rather than reacting only to current observations, an optimal agent maintains and updates a belief over possible future states and selects actions that maximize expected outcomes based on these predictions. This idea underpins modern model-based learning, where agents build internal models of how the world evolves and use them to plan ahead.

#### 2.1.2.1 Formal Problem Setup

The following formulation provides a compact mathematical abstraction of behavioural cloning, induced distribution shift, and SAILOR-style residual planning, adapted from the algorithmic descriptions in the cited literature [4], [5], [7], [9].

We model robotic imitation learning as a partially observed sequential decision-making problem with observation space $O$, action space $A$, and latent state space $Z$. At time step $t$, the agent receives an observation $o_t \in O$, executes an action $a_t \in A$, and induces future observations through partially observed dynamics [9].

Given an expert demonstration dataset

$$D = \{(o_t^i, a_t^i)\},$$

behavioural cloning learns a policy $\pi_\theta(a \mid o)$ by minimizing a supervised imitation objective of the form

$$L_{BC}(\theta) = \mathbb{E}_{(o,a)\sim D}[\ell(\pi_\theta(o), a)].$$

This formulation makes the distribution-shift problem explicit: training data are drawn from the expert-induced distribution, whereas at test time future observations depend on the learner's own previous actions [5], [7]. As a result, small action errors can accumulate over time and move the policy into states that are weakly represented in the demonstration dataset.

SAILOR augments a base policy with a learned latent world model, reward model, and critic [4]. At time step $t$, the base policy proposes a nominal action sequence

$$a_{t:t+H-1}^{\text{base}},$$

and planning is performed over residual corrections

$$\Delta_{t:t+H-1}.$$

If $z_t \in Z$ denotes the current latent state, imagined future latent states are generated by the learned dynamics model

$$z_{t+k+1} = f_\phi\left(z_{t+k}, a_{t+k}^{\text{base}} + \Delta_{t+k}\right).$$

Candidate residual sequences are then evaluated using a learned reward model and terminal value estimate, giving an objective of the form

$$J(\Delta) = \sum_{k=0}^{H-1} \hat{r}_\psi\left(z_{t+k}, a_{t+k}^{\text{base}} + \Delta_{t+k}\right) + \hat{V}_\omega(z_{t+H}).$$

The planner selects a high-scoring correction sequence and executes only the first corrected action before replanning.

### 2.1.3 Model-Based Reasoning and World Models

As discussed in the preceding sections, BC and other imitation-based methods are inherently reactive, relying on static expert data without an understanding of how actions influence future states. This lack of predictive reasoning limits their ability to recover from errors or handle novel situations. Model-based reasoning addresses this limitation by enabling agents to learn an internal model of the environment's dynamics, allowing them to predict how the environment will evolve in response to actions and to plan accordingly.

A world model is a learned internal simulator that approximates the transition dynamics of the real environment. Given the current state and an action, it predicts the next state, allowing the agent to reason about the consequences of potential actions before executing them. In robotic manipulation, world models make it possible to simulate future motions and outcomes without requiring real-world interaction, improving both safety and data efficiency [10].

Because raw sensory inputs such as images or proprioceptive signals are high-dimensional and redundant, modern world models operate in a latent space, a compact learned representation that captures the essential information needed for prediction and control. The latent state, often denoted $\boldsymbol{z_t}$, summarizes the relevant features of the environment at a given time step. By working in this compressed space, the agent can perform efficient long-horizon reasoning without reconstructing or processing full sensory inputs at every step.

The Recurrent State-Space Model (RSSM) is a widely adopted architecture for learning latent dynamics, introduced in the PlaNet framework [11] and later used and refined in the Dreamer family of algorithms [12], [14]. The RSSM combines deterministic and stochastic components to capture both predictable structure and uncertainty in the environment.

- The deterministic component is implemented using a Recurrent Neural Network (RNN), such as a Gated Recurrent Unit (GRU). RNNs are designed to process sequential data by maintaining an internal hidden state that is updated at each time step. This hidden state acts as a memory, allowing the model to retain information from previous observations and actions. In the context of world models, this means the RSSM can encode temporal dependencies, learning how past interactions influence current dynamics rather than treating each step as independent.
- The stochastic component captures uncertainty and variability in the system by sampling from a learned probability distribution over latent states.

At each time step, the RSSM takes the previous latent state and current action as input and predicts the next latent state. Formally, the dynamics function $\boldsymbol{p}(\boldsymbol{z_{t+1}} \mid \boldsymbol{z_t}, \boldsymbol{a_t})$ defines how latent variables evolve. The RSSM therefore functions as a compact world simulator, generating sequences of imagined latent states that reflect possible future trajectories of the environment [11], [12].

By propagating latent states forward through the learned dynamics, the agent can evaluate alternative action sequences without interacting with the real world. A reward or value model can then be applied to these imagined trajectories to estimate their expected performance and identify the most promising actions. This process enables planning and recovery when the robot encounters unfamiliar states, as the model can simulate likely outcomes and select corrective actions before committing to them in the real environment.

The flexibility of the RSSM has made it a core building block in several advanced model-based learning systems, including DreamerV1–V4 and TD-MPC [12] – [14], [13]. In these systems, the world model serves as the predictive backbone: policies and value functions are trained entirely in latent space using imagined rollouts generated by the RSSM. DreamerV2 introduced categorical latent variables that improved representation capacity, DreamerV3 enhanced generalization and stability across diverse environments, and the recent DreamerV4 [14] extended the framework to

handle larger-scale, multi-domain control with improved optimization and self-improving world model learning. These advances demonstrate the scalability and robustness of RSSM-based reasoning for real-world control.

### 2.1.4 Dreamer and Latent Imagination-Based Planning

The Dreamer family of algorithms demonstrates how learned world models can be used not only for prediction but also for decision-making through imagination. Building on the principles introduced in Section 2.2.3, Dreamer integrates world, reward, and value models into a unified architecture that allows an agent to plan and optimize actions entirely within a learned latent dynamics space. Rather than relying on real environment rollouts to improve its behaviour, Dreamer trains its policy and value functions on trajectories imagined inside the latent space of its Recurrent State-Space Model (RSSM) [12], [14].

At the core of Dreamer are four interacting components: a world model, a reward model, a value model (critic), and a policy model (actor). The world model learns to predict how latent states evolve given actions. It encodes sensory inputs, such as images or proprioceptive signals, into a compact latent representation and models how these latent states transition over time. The reward model estimates the immediate reward associated with each latent state or transition, providing a measure of task progress within imagination. The value model predicts the expected cumulative return from a latent state, allowing long-term evaluation of imagined trajectories. Finally, the policy model generates actions that maximize predicted value, effectively learning to act optimally within the simulated latent environment [12], [11].

Dreamer's learning process unfolds through three main stages.

1. The world model, reward model, and value model are first trained from real data collected through environment interaction. The objective is to minimize reconstruction and prediction errors, ensuring that the latent dynamics accurately capture environment behaviour.

2. Once the models are trained, the world model is used to generate imagined trajectories by unrolling latent transitions conditioned on hypothetical action sequences. This process is known as latent imagination, where the agent explores possible futures entirely within its learned representation space.

3. The actor and critic are updated using the imagined rollouts. The critic learns to estimate returns from the imagined trajectories using temporal-difference learning, and the actor is optimized to select actions that maximize these predicted returns. Because gradients are propagated through the imagined trajectories, both models improve without requiring additional environment interaction.

By reasoning about long-term outcomes in its latent space, Dreamer can anticipate the consequences of its actions and select high-value sequences without costly physical exploration. The approach has proven highly data-efficient, achieving strong results on continuous-control benchmarks from image inputs with orders of magnitude fewer environment steps than model-free methods [12], [14].

Successive versions of Dreamer have refined these principles. DreamerV2 introduced categorical latent variables to improve representation richness and stability in long-horizon prediction. DreamerV3 demonstrated scalability across diverse domains using improved normalization, value estimation, and regularization strategies. The recent DreamerV4 extended this framework to multi-domain control tasks, incorporating self-improving world models that continually refine their predictive accuracy and policy performance [14]. Collectively, these versions show how learning within imagination can generalize across environments and tasks, establishing Dreamer as a foundation for model-based reasoning in robotics and control.

#### 2.1.5 Learning to Search and Planning in Latent Spaces

As discussed in Section 2.2.4, world models such as those used in Dreamer enable agents to reason about future outcomes through internal imagination. Extending this principle from training into execution gives rise to Learning to Search (L2S) and planning in latent spaces, where the agent performs active, test-time reasoning to refine or correct its own policy. These approaches link model-based prediction with online decision-making, allowing policies not only to act from learned mappings but also to adapt dynamically by searching for improved actions when errors or uncertainty arise.

The concept of L2S originates in structured prediction and imitation learning, where sequential decision-making is treated as a search problem over trajectories of actions and states. Early formulations such as SEARN (Search-based Structured Prediction) and AggreVaTe (Aggregate Values to Imitate) framed policy learning as the iterative refinement of a search policy using expert

feedback [5], [15]. Instead of imitating single actions in isolation, these methods trained a policy to minimize long-term cost by evaluating how each decision influenced subsequent states. Each iteration alternated between executing the current policy, observing where it diverged from expert trajectories, and updating the policy using the expert's corrections. The key insight was that robust sequential policies must learn from their own induced state distributions rather than depending solely on the expert's. This closed-loop formulation of imitation learning is directly relevant to robot control, where small mistakes can cascade if not corrected through predictive reasoning.

In continuous control, the L2S framework converges naturally with Model Predictive Control (MPC), a long-established approach in optimal control theory. MPC maintains an explicit model of system dynamics and performs short-horizon optimization at every time step. The controller predicts the future evolution of the system over a finite horizon, identifies an action sequence that minimizes a cost function, executes only the first action, and then replans from the new state. This plan–execute–replan cycle produces adaptive, receding-horizon behavior capable of responding to disturbances or modeling errors in real time [9], [16]. In robotics, MPC has proven effective for tasks involving contact, constraints, or nonlinear dynamics because it balances immediate performance with long-term stability.

A recent extension of these ideas is residual planning, where planning is performed not from scratch but as a correction over a base policy. Instead of generating entirely new action sequences, the planner searches for residuals ($\Delta_{t:t+k}$) that adjust the nominal actions proposed by the base policy. This residual approach provides a balance between learned priors and adaptive control: the base policy offers a good initial guess grounded in demonstration data, while the residual search corrects errors that arise from distribution shift or imperfect modeling. By combining these components, the system achieves both efficiency and robustness, maintaining expert-like behavior even when encountering unfamiliar conditions [4].

Searching directly in the high-dimensional observation space, such as images or proprioceptive readings, is computationally intractable. Instead, modern learning-based controllers perform latent-space search, operating within the compact representations learned by world models. As described in Section 2.2.3, latent states summarize the task-relevant structure of the environment, allowing the controller to unroll imagined trajectories efficiently. During search, the agent samples candidate action sequences, propagates them through the learned latent dynamics model, and

evaluates the resulting trajectories using a reward or value model. Sequences with higher predicted returns correspond to better corrective plans. Because the latent space captures essential features while discarding sensory redundancy, search and optimization can proceed orders of magnitude faster than direct planning in the raw sensory domain. This approach enables real-time decision-making and recovery in robotics, where computational efficiency is critical.

In the context of model-based control, latent-space search unifies learning and planning. The learned dynamics model functions as the plant, the reward or value function serves as the cost model, and the policy acts as the optimizer. Together, these components implement a self-contained model predictive controller that operates entirely in a learned representation space. By continually planning, executing, and replanning within this space, the agent achieves adaptive behavior analogous to classical MPC while leveraging the representational power of deep world models. Importantly, these searches can be guided not by hand-crafted cost functions but by learned reward or critic models that encode expert-like preferences.

### 2.1.6 The SAILOR Framework

The SAILOR framework (Searching Across Imagined Latents Online for Recovery) [4] unites imitation learning, world-model reasoning, and search-based planning into a single architecture for robust control. It was developed to address the central weakness of behavioural cloning: its inability to recover once the policy drifts off distribution. SAILOR enables a robot to imagine possible futures, evaluate them with learned models, and execute corrective actions that restore expert-like behaviour without requiring new demonstrations.

#### 2.1.6.1 Objective: Robust Imitation through Latent-Space Search

SAILOR's objective is to transform static imitation into adaptive control. Rather than directly executing the base policy's predicted actions, SAILOR treats that policy as an initial proposal and searches for small corrective action sequences within a learned latent world model. Candidate plans are imagined forward in latent space, scored by learned reward and value functions, and the best plan's first action is executed before replanning. This process functions as MPC performed entirely in latent space, giving the system the ability to anticipate and recover from distribution shift while maintaining the efficiency of imitation learning.

#### 2.1.6.2 Core Components

**Base Policy ($\pi_{base}$)**

The base policy provides an initial k-step action plan given the current observation. In the original implementation, this is a Diffusion Policy [6], trained by behavioural cloning on expert demonstrations to generate temporally consistent motion sequences from visual and proprioceptive inputs. Although SAILOR uses diffusion policies for simplicity, the framework is agnostic to architecture and can incorporate pretrained visuomotor or foundation models as priors. The base policy establishes a starting point that the planner refines through search.

**World Model (WM)**

The world model predicts the evolution of the environment in a compact latent space. SAILOR adopts the RSSM architecture as implemented in DreamerV3 [14], which builds upon earlier RSSM formulations originally introduced in PlaNet [11]. RSSM combines deterministic recurrence with stochastic latent variables to model both predictable structure and uncertainty in environment dynamics.

The world model comprises three components:

1. an encoder that maps recent observations and actions into a latent state representation,
2. a latent dynamics model that predicts future latent transitions, and
3. a decoder that reconstructs observations from latent states to stabilize learning.

The world model is trained on hybrid data consisting of both expert demonstrations (D) and rollouts from the learner (B). This ensures that it remains accurate not only near expert trajectories but also within the state distributions induced by the agent's own policy during execution. Prior work on no-regret imitation learning shows that training under these induced distributions yields bounded performance degradation in the real environment, mitigating compounding error under distribution shift [5].

**Reward Model (RM)**

The reward model evaluates how "expert-like" imagined latent trajectories are. It acts as a learned reward function that guides search and planning without requiring task-specific supervision. SAILOR trains this model using the moment-matching loss introduced by Swamy et al. [17], which compares statistical features of expert and learner trajectories. Rather than predicting a scalar reward directly from ground truth, the moment-matching objective encourages the model to

assign higher scores to latent states whose distributions match those of the expert and lower scores to those that diverge.

Intuitively, this loss ensures that the reward model captures the distributional structure of expert behaviour rather than specific action labels. It effectively treats the problem as distinguishing between “expert-like” and “non-expert-like” latent transitions, learning to value states that reflect expert consistency, precision, and task success. Over the course of SAILOR training, this process helps the method generalize to unfamiliar conditions, as the reward model is continuously updated to reflect the evolving competence of the learner. A gradient penalty [19] is added during training to stabilize this adversarial-style optimization and prevent the model from overfitting to narrow regions of latent space.

**Critic / Value Network (V)**

The critic estimates the long-term value of imagined trajectories, allowing the planner to evaluate sequences beyond immediate rewards. It plays a similar role to the value model in Dreamer, but here it supports planning rather than direct policy optimization. Specifically, the critic provides a terminal cost estimate for truncated rollouts imagined in the world model, allowing SAILOR to assess not only immediate reward predictions but also the expected cumulative outcome over time.

SAILOR uses an ensemble of five critics [20], [21], to enhance stability and uncertainty awareness. Each critic is trained on bootstrapped targets derived from the reward model’s evaluations, and during planning, the mean prediction of two randomly sampled critics is used while an uncertainty penalty proportional to their disagreement is subtracted. This ensemble design reduces overestimation bias and prevents the planner from relying on uncertain trajectories. The result is a more reliable evaluation function that guides search toward robust, high-value corrections rather than unstable local optima.

Collectively, the world, reward, and critic models enable latent-space reasoning, the ability to simulate possible futures, measure their expected quality, and plan accordingly. Together with the base policy, they form a self-contained predictive control system capable of online adaptation.

#### 2.1.6.3 Operational Phases

SAILOR’s training and execution are organized into three distinct but interconnected phases, each contributing to the system’s ability to both act and learn effectively.

**Phase I: Warm Start**

In this initial stage, SAILOR learns its internal predictive and evaluative models: the world model (WM), reward model (RM), and critic (V). These are trained on a hybrid dataset composed of expert demonstrations (D) and rollouts from the base policy (B). This co-training ensures that the models remain accurate not only around expert trajectories but also within the regions the learner is likely to visit during its own execution.

The goal of this phase is to align the agent's internal representation with both expert and learner distributions. This hybridization is critical because, as demonstrated by Ross and Bagnell [5], a model trained only on expert data can fail once the learner's actions induce new, unseen states. By contrast, hybrid training allows the world model to remain valid even when the learner begins to drift, providing a stable foundation for planning and recovery.

**Phase II: Online Fine-Tuning**

After initialization, SAILOR begins executing its latent-space search planner in the environment. During this process, it continuously collects new on-policy rollouts that are added to its replay buffer. The world model, reward model, and critic are periodically fine-tuned on this evolving dataset to ensure that their predictions remain consistent with the states the current policy visits.

This phase makes SAILOR an adaptive system. Instead of freezing its models after offline training, it updates them alongside the evolving behaviour of the agent. In practice, this allows SAILOR to refine its understanding of dynamics and rewards as it gains experience, similar to continual model-based reinforcement learning.

**Phase III: Expert Iteration (Policy Distillation)**

Over time, SAILOR's planner discovers latent trajectories that yield higher predicted rewards and values. In this phase, those imagined or executed high-performing behaviours are distilled back into the base policy. The planner's corrected action sequences are relabelled as new "expert" examples, and the base policy is fine-tuned using behavioural cloning.

This process mirrors the DAgger principle [5], where the agent iteratively refines its policy using improved data from its own experience. However, unlike classical DAgger, SAILOR does not require external expert feedback. Its world, reward, and critic models collectively act as an internal

teacher. Through this self-distillation, the base policy absorbs the planner's corrections, leading to a progressively more capable imitation model that requires less online search over time.

#### 2.1.6.4 Algorithmic Intuition

The SAILOR paper formalizes these ideas through two algorithms that together describe inference and learning.

**Algorithm 1 — SAILOR (Inference)**

**Algorithm 1** SAILOR (Inference)

**Input:** Base Policy $\pi^{\text{base}}$, World Model WM, Reward Model RM, Critic Network V, Obs. $o_t$,
Sample nominal plan from base: $a^{\text{base}}_{t:t+k} \sim \pi^{\text{base}}(o_t)$.
**for** iteration $j$ in $1 \dots J$ **do**
  // Can perform in parallel
  **for** residual plan $n$ in $1 \dots N$ **do**
    Sample $\Delta^n_{t:t+k} \sim \mathcal{N}(\mu^j, \sigma^j)$.
    Execute plan in WM: $z^n_{t:t+k} \sim \texttt{WM}(o_t, a^{\text{base}}_{t:t+k} + \Delta^n_{t:t+k})$.
    Compute $\hat{Q}$s: $\hat{Q}^n \leftarrow \sum_{h=0}^{k-1} \gamma^h \texttt{RM}(z^n_{t+h}) + \gamma^k \texttt{V}(z^n_{t+k})$.
  **end for**
  // Update mean and std
  $\mu^{j+1}, \sigma^{j+1} \leftarrow \texttt{MPPI_update}(\mu^j, \sigma^j, \hat{Q}^{1:N}, \Delta^{1:N}_{t:t+k})$.
**end for**
**Return** corrected plan $a^\star_{t:t+k} \sim \mathcal{N}(a^{\text{base}}_{t:t+k} + \mu^J, \sigma^J)$.

**Figure 2.1: Algorithm 1 SAILOR (Inference) [4]**

Algorithm 1 describes SAILOR's test-time reasoning loop. At each control step:

1. The base policy proposes a nominal k-step plan $a^{base}_{t:t+k}$.
2. A population of residual plans $\Delta_{t:t+k}$ is sampled from a Gaussian distribution parameterized by mean μ and covariance σ.
3. Each candidate plan is rolled out through the world model to produce imagined latent trajectories $z_{t:t+k}$.
4. The reward model assigns per-step rewards, and the critic predicts the terminal value of the final latent state.
5. These scores are combined to form a total quality estimate for each rollout.

6. Using Model Predictive Path Integral (MPPI) control [18], the mean and covariance of the residual distribution are updated toward higher-scoring samples.

7. After several iterations, the corrected plan $a^*_{t:t+k}$ is selected, and only its first action is executed before replanning.

This inference loop functions as latent-space model predictive control. The MPPI update acts as a gradient-free optimizer that progressively re-weights better-performing samples, focusing search around locally optimal corrections.

**Algorithm 2 — SAILOR (Training)**

**Algorithm 2** SAILOR (Training)

**Input:** Base Policy $\pi^{\text{base}}$, Expert Demos $\mathcal{D}$
// Phase I: Warm Start
Collect on-policy rollouts: $\mathcal{B} \leftarrow \{(o_t, a_t) \sim \pi^{\text{base}}\}$.
// Co-train components on hybrid data
$\texttt{WM} \leftarrow \text{argmin}_{\texttt{WM}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{WM}), \texttt{RM} \leftarrow \text{argmin}_{\texttt{RM}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{RM}), \texttt{V} \leftarrow \text{argmin}_{\texttt{V}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{V})$.
// Phase II: Online Fine-Tuning
**for** iteration $j$ in $1 \ldots J$ **do**
  Collect on-policy rollouts: $\mathcal{B} \leftarrow \mathcal{B} \cup \{(o_t, a_t) \sim \texttt{SAILOR}\}$.
  $\texttt{WM} \leftarrow \text{argmin}_{\texttt{WM}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{WM}), \texttt{RM} \leftarrow \text{argmin}_{\texttt{RM}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{RM}), \texttt{V} \leftarrow \text{argmin}_{\texttt{V}}\, \ell(\mathcal{D}, \mathcal{B}, \texttt{V})$.
  // Phase III: Expert Iteration
  **if** $j \,\%\, m == 0$ **then**
    Relabel $\mathcal{B}^{\text{distill}} \leftarrow \{(o_t, \texttt{SAILOR}(o_t)) | o_t \in \mathcal{B}\}$.
    Distill $\pi^{\text{base}} = \text{argmin}_{\pi}\, \ell(\mathcal{B}^{\text{distill}}, \pi)$.
  **end if**
**end for**
**Return** $\pi^{\text{base}}$, WM, RM, V.

**Figure 2.2: Algorithm 2 SAILOR (Training) [4]**

Algorithm 2 defines the full training loop:

1. Warm Start: Collect initial rollouts and jointly train WM, RM, and V on expert and learner data.

2. Online Fine-Tuning: Execute SAILOR's planner to collect new on-policy rollouts and continually update WM, RM, and V.

3. Expert Iteration: Every m iterations, relabel buffer trajectories with SAILOR's corrected actions and perform behavioural cloning updates to distill these improved behaviours back into $\pi_{base}$.

This process couples model learning and policy refinement in a feedback loop where the planner uses the learned models to find better behaviours, and the base policy absorbs those behaviours for future inference.

#### 2.1.6.5 Model Predictive Path Integral Control and Policy Improvement

The MPPI method [18] is the optimization engine behind SAILOR's search process. MPPI belongs to the class of sampling-based stochastic optimizers, closely related to the Cross-Entropy Method (CEM) but smoother and gradient-free.

At each planning step, MPPI maintains a Gaussian distribution over candidate residual plans. After simulating these candidates in the world model, each is scored by the reward and critic networks. MPPI then re-weights the samples exponentially by their scores, updating the distribution's mean and variance so that it gradually concentrates around the best-performing trajectories.

In intuitive terms, MPPI works like a "soft selection" process: high-reward imagined plans pull the sampling distribution toward better directions in action space, while low-reward ones fade out. This enables the planner to refine its action proposals iteratively without backpropagating through the model or requiring differentiable dynamics.

Within SAILOR, this process effectively performs Model-Based Monte Carlo Policy Improvement (MMPI). The improved plan distribution found by MPPI represents a locally optimized correction to the base policy. Over time, these optimized residuals are distilled back into the policy itself, bridging the gap between online planning and offline learning.

#### 2.1.6.6 Outcome and Significance

By combining predictive world modeling, reward discrimination, and sampling-based latent-space planning, SAILOR achieves several key outcomes:

- Recovery from distribution shift: The planner performs test-time corrections when the base policy drifts from expert behaviour.

- Data efficiency: Learning and planning occur primarily through imagined rollouts, reducing the need for real-world data.
- Self-improvement: High-reward imagined trajectories are distilled back into the policy, enabling continual refinement.
- Uncertainty-aware robustness: Ensemble critics penalize uncertainty, and moment-matching reward learning provides smoother, more stable optimization.

Together, these mechanisms make SAILOR a powerful and scalable framework for robust imitation learning, capable of both online recovery and long-term adaptation.

### 2.1.7 Limitations of SAILOR and Motivation for Stronger Base Policies

While SAILOR demonstrates strong recovery and self-improvement, its overall effectiveness depends on the representational quality of its base policy. In the original framework, this base policy was a diffusion model trained on limited demonstrations. Diffusion policies learn to generate smooth, temporally consistent action sequences that model multimodal behaviour within their training distribution [6]. They perform well within that distribution but lack semantic grounding and generalization to unseen conditions. Because their understanding is tied to the demonstrated trajectories, they struggle when the robot encounters novel objects, scene variations, or new task goals.

This limitation arises not from SAILOR's search process but from the priors provided by its base policy. Pretrained multimodal models, particularly Vision–Language–Action (VLA) models, offer a potential remedy. These models are trained on large and diverse datasets that couple perception, language, and control, allowing them to develop structured visuomotor and semantic representations that extend beyond individual tasks. Integrating such pretrained priors into SAILOR could provide richer representations of scenes and goals, improving both recovery and generalization.

### 2.1.8 Pretrained Base Policies and Foundation Models in Robotics

Foundation models in robotics extend the principles of large-scale multimodal learning to embodied control. Their development follows a natural progression:

1. Large Language Models (LLMs) capture abstract reasoning and world knowledge from text.
2. Vision–Language Models (VLMs) link textual and visual understanding, grounding language in perception [22].
3. Vision–Language–Action (VLA) models add control generation, mapping perception and language directly to robot actions [23], [24].

A VLA typically consists of two main components: a vision–language encoder and an action head. The encoder processes camera inputs and optional textual goals into a joint latent representation that captures both scene context and task intent. The action head then maps this representation to continuous control outputs, such as end-effector displacements, rotations, and gripper states. These commands are produced at high frequency to ensure smooth and responsive motion. In practical terms, each "action" represents a single control vector that specifies how the robot should move at a given time step, allowing continuous control at rates comparable to real-world execution.

**$\pi_0$ and $\pi_0$-Droid**

$\pi_0$ (Pi-Zero) is a recent VLA model that integrates a pretrained vision–language backbone with a flow-matching action head [2]. It uses conditional flow matching to generate continuous action distributions. This technique trains a neural network to learn a smooth, deterministic transformation that maps random noise to expert actions, conditioned on the robot's current visual observation and language goal. Unlike diffusion-based models, which generate actions by progressively denoising samples over many iterations, flow matching defines a continuous flow that directly transports samples from a simple base distribution (such as Gaussian noise) to the target expert action distribution in a single forward pass. This makes action generation faster and more stable while maintaining the diversity and richness of diffusion-based methods. As a result, $\pi_0$ can output coherent control trajectories at approximately 50 Hz, supporting smooth, real-time manipulation.

$\pi_0$ is trained on large-scale multimodal data combining Internet videos, simulated demonstrations, and robot interaction datasets, which gives it broad semantic and visuomotor generalization across diverse objects and tasks.

$\pi_0$-Droid fine-tunes $\pi_0$ on the DROID dataset [1], which contains over 76 000 real-world manipulation trajectories across hundreds of tasks and multiple robot platforms. Fine-tuning aligns $\pi_0$'s general visuomotor and semantic priors with the DROID hardware stack, which is built around a Franka Panda arm and gripper and is therefore well aligned with the Franka-based RoboMimic benchmark setting [1], [3], [29], [31], [32].

**Comparative Perspective**

Compared with diffusion policies, $\pi_0$ and related VLA models such as RT-2 [22], OpenVLA [23], and Helix [24] provide stronger semantic grounding and more transferable visuomotor priors. Their large, heterogeneous training data enable generalization across tasks and embodiments rather than relying on narrow demonstration distributions; however, these models remain feedforward controllers that map perception directly to action without explicit planning or test-time search. Integrating such pretrained priors within SAILOR's latent-space search could therefore combine the generalization of foundation models with the adaptability of predictive planning.

## 2.2 Comparative Review

The next part of this review examines complementary methods that refine imitation policies beyond static learning, including reinforcement learning on top of imitation, search-augmented control, and policy distillation techniques that bridge planning and execution.

### 2.2.1 Learning-to-Search (L2S) Methods

Learning-to-Search (L2S) methods refine imitation-learned policies by evaluating decisions at the level of trajectories rather than individual actions. Unlike behavioural cloning, which optimizes per-step imitation loss, L2S algorithms explicitly account for how early decisions affect future states and cumulative performance under the learner's own induced distribution [5], [15].

Classical L2S approaches such as SEARN, DAgger, and AggreVaTe demonstrated that robustness in sequential decision making requires iterative correction during execution rather than static imitation. These methods provide strong theoretical guarantees but depend on frequent expert supervision, limiting their practicality in robotic manipulation where expert queries are costly or unsafe.

Recent work shifts L2S away from explicit expert correction toward internal evaluation using learned models, critics, or short-horizon planning. In continuous control, this trend appears in

model-based and MPC-style methods that refine policy outputs through trajectory search guided by learned value or cost functions [13], [16]. While often framed as reinforcement learning or planning, these methods share the core L2S principle of trajectory-level refinement under execution-time distributions.

SAILOR belongs to this modern class of L2S methods. It retains the L2S objective of correcting behavior under distribution shift but replaces expert supervision with reward-guided latent search using learned world and value models [4]. Corrective trajectories are generated through imagination and distilled back into the policy, mirroring dataset aggregation without requiring human relabeling.

Thus, SAILOR can be understood as an expert-free, model-based instantiation of L2S, positioned between classical interactive imitation learning and reinforcement learning-based policy refinement. This framing distinguishes it from reward-driven fine-tuning approaches and motivates the following sections with alternative refinement strategies.

### 2.2.2 Reinforcement Learning on Top of Imitation Learning (RL-on-IL)

A widely studied approach for improving imitation-learned policies is to refine them using reinforcement learning after an initial imitation phase. In this two-stage approach, a policy is first trained using behavioural cloning or a related imitation objective, and then further optimized through reinforcement learning to maximize a task-specific reward. This IL to RL pipeline aims to address the core weakness of imitation learning: its inability to optimize long-term outcomes or handle rare, safety-critical states that are underrepresented in demonstration data.

The motivation for combining imitation learning with reinforcement learning is that the two methods provide complementary learning signals. Imitation learning offers strong sample efficiency and produces human-like behaviour by leveraging large datasets of expert demonstrations, but it does not explicitly encode notions of safety, optimality, or long-term performance. Reinforcement learning, by contrast, optimizes cumulative reward in a closed-loop setting and can learn causal relationships between actions and outcomes, enabling correction under distribution shift. However, RL is often unstable, sample-inefficient, and highly sensitive to reward design, particularly in real-world robotic systems [15].

A representative and influential example of this hybrid approach is the IEEE IROS 2023 paper "Imitation Is Not Enough: Robustifying Imitation with Reinforcement Learning for Challenging Driving Scenarios" by Lu et al. [25]. In this work, the authors train an autonomous driving policy on over 100,000 miles of real-world human driving data using behavioural cloning and then refine it using reinforcement learning with a simple safety-oriented reward. Their method, termed BC-SAC (Behavior-Cloned Soft Actor-Critic), combines a standard Soft Actor-Critic reinforcement learning objective with an imitation loss to preserve expert-like behaviour while improving robustness. The reward encodes collision avoidance and off-road penalties, while imitation learning maintains naturalistic driving behaviour. BC-SAC significantly reduces failure rates in rare and difficult driving scenarios, achieving over a 38 percent reduction in safety events compared to imitation-only baselines.

This study illustrates several important properties of RL-on-IL methods. First, reinforcement learning is particularly effective in regions of the state space where demonstration data is sparse. Lu et al. explicitly show that imitation learning performs well in low-difficulty scenarios that are well covered by data, while reinforcement learning provides outsized gains in high-difficulty, long-tail scenarios where expert data alone is insufficient [25]. Second, RL enables closed-loop correction by exposing the policy to its own induced state distribution, mitigating covariate shift and compounding errors that arise in open-loop imitation. Third, the use of simple, safety-focused rewards reduces but does not eliminate the challenge of reward design, as balancing imitation fidelity and reward optimization requires careful tuning.

Despite these benefits, RL-on-IL approaches have notable limitations. Reinforcement learning requires extensive environment interaction, which can be expensive, slow, or unsafe in real robotic systems. Even when trained in simulation, transferring RL-refined policies to real hardware remains challenging due to modeling errors and reward misspecification. Moreover, RL fine-tuning can degrade behaviours learned from demonstrations if the reward function conflicts with expert preferences, leading to unnatural or overly conservative policies. These trade-offs are evident in the driving domain, where RL-only baselines often achieve safety at the cost of realism or progress [25].

From a conceptual perspective, RL-on-IL methods refine policies by optimizing an explicit reward function through real or simulated interaction. This distinguishes them from Learning-to-Search

approaches discussed in Section 2.2.1, which focus on correcting behaviour through trajectory-level evaluation rather than reward maximization. It also differentiates them from SAILOR. While SAILOR shares the goal of improving robustness beyond static imitation, it does not perform reinforcement learning over environment interactions. Instead, SAILOR uses a learned reward model that evaluates expert-likeness and performs corrective planning through latent imagination, avoiding the need for explicit reward engineering and large-scale RL rollouts [4].

In this sense, reinforcement learning on top of imitation learning represents a powerful but costly strategy for robustness, particularly effective in domains where safety rewards can be clearly defined and extensive interaction is feasible. SAILOR achieves a similar objective through internal, model-based search, positioning it as an alternative to RL-on-IL that emphasizes data efficiency and expert-free refinement rather than reward-driven policy optimization.

### 2.2.3 Distillation of Planning Results into Policies

A common strategy for refining imitation-learned or planner-based controllers is to distil the results of planning into fast, reactive policies. In these approaches, a computationally expensive planner or search procedure is used to generate high-quality action trajectories, which are then used as supervision to train a policy that can be executed efficiently at test time. This allows systems to benefit from planning during training while maintaining low-latency control during deployment.

This idea appears prominently in expert iteration methods, where planning and learning are tightly coupled. A planner evaluates candidate action sequences and selects high-quality trajectories, and a policy is subsequently trained to imitate these decisions. Over successive iterations, the policy improves, which in turn improves the planner's proposals. This approach was popularized in reinforcement learning through Expert Iteration and AlphaZero-style methods, where Monte Carlo Tree Search provides supervision for a neural policy that is later executed without search [26].

In continuous control and robotics, similar ideas appear in model-based planning frameworks. Temporal Difference Model Predictive Control (TD-MPC) combines short-horizon planning in a learned latent dynamics model with value estimation and optionally distils the resulting plans into a policy network for faster execution [13]. Related approaches train policies to imitate the outputs of MPC or sampling-based planners, effectively compiling planning behaviour into a feedforward controller. These methods improve efficiency and deployability, particularly in real-time systems where repeated planning may be computationally prohibitive.

However, distillation introduces an inherent trade-off. While distilled policies execute quickly, they typically lose the ability to adapt online, as the planning process that generated the corrective behaviour is no longer active at test time. Once the policy deviates from its training distribution, it cannot revise its decisions without reintroducing planning or search.

SAILOR adopts a hybrid approach to distillation that preserves adaptivity. Rather than fully replacing planning with a distilled policy, SAILOR continues to perform latent-space search during execution while periodically distilling high-quality imagined or executed trajectories back into the base policy [4]. This expert-iteration-style loop improves the policy over time without eliminating test-time correction. As a result, SAILOR combines the efficiency benefits of distillation with the robustness of online planning, avoiding the rigidity that often accompanies purely distilled controllers.

### 2.2.4 Vision–Language–Action (VLA) Models as Strong Imitation Priors

Vision–Language–Action (VLA) models represent a recent class of foundation models that directly map visual observations and natural-language instructions to continuous robot actions. By extending vision–language models with action-generation heads, VLAs unify perception, semantic understanding, and control within a single learned policy. Unlike classical imitation learning methods that are trained on narrow task-specific datasets, VLAs are trained on large, heterogeneous multimodal corpora, enabling generalization across tasks, environments, and embodiments.

Several influential VLA models illustrate architectural choices. RT-1 introduced the Robotics Transformer, which learns a generalist control policy by training a transformer model on multi-task robot demonstrations using image observations and language instructions [27]. RT-2 extended this approach by incorporating web-scale vision–language pretraining, allowing semantic concepts learned from Internet data to transfer into robotic control and enabling improved generalization to unseen tasks and objects [22]. OpenVLA proposed an open-source VLA framework trained on large multimodal datasets with a focus on accessibility and cross-robot generalization [23]. SmolVLA demonstrated that compact VLA models, with significantly fewer parameters than proprietary systems, can still achieve strong performance on standard manipulation benchmarks when trained on curated multimodal datasets such as LeRobot [28]. Helix further explored VLA

architectures for high-degree-of-freedom control, including humanoid manipulation, using a dual-system design that separates high-level semantic reasoning from low-level motor execution [24].

$\pi_0$ represents a distinct architectural direction within the VLA family. Rather than using autoregressive token prediction or diffusion-based action generation, $\pi_0$ employs conditional flow matching to generate continuous action distributions conditioned on visual observations and language goals [2]. This approach enables fast, stable action generation at high control frequencies, making $\pi_0$ suitable for real-time manipulation. $\pi_0$ is trained on a mixture of large-scale Internet vision–language data and robot interaction datasets, allowing it to acquire broad semantic grounding alongside visuomotor competence. $\pi_0$-Droid is obtained by fine-tuning $\pi_0$ on the DROID dataset, aligning the model with the DROID hardware stack built around a Franka Panda arm and gripper, which is well matched to the Franka-based RoboMimic benchmark setting [1], [3], [31], [32].

However, despite their strong generalization capabilities, existing VLA models remain fundamentally feedforward controllers. They map observations and goals directly to actions without explicit test-time reasoning, planning, or recovery mechanisms. When execution deviates from familiar states due to noise, perturbations, or unexpected interactions, VLAs lack internal mechanisms to evaluate alternative futures or correct their behaviour. This limitation mirrors the distribution-shift problem observed in behavioural cloning, albeit at a larger scale.

### 2.2.5 Summary and Positioning of This Thesis

The policy refinement strategies reviewed in this section highlight complementary strengths and limitations. L2S methods correct imitation policies by reasoning over trajectories under the learner's induced distribution but traditionally rely on expert supervision. RL-on-IL improves robustness and long-term performance through reward optimization but introduces significant sample complexity and reward-design challenges. Distillation-based approaches improve efficiency by compiling planning outcomes into reactive policies, often at the cost of reduced adaptivity at test time. VLA models provide powerful generalization through large-scale multimodal pretraining but lack mechanisms for test-time correction.

These methods expose a fundamental divide in current robot learning systems. Planning-based approaches support recovery and adaptation but depend on relatively narrow priors learned from

limited data. Foundation models generalize across tasks and environments but operate without explicit reasoning or recovery once deployed.

This thesis is positioned at the intersection of these two directions. By integrating $\pi_0$-Droid, a pretrained VLA model, as the base policy within the SAILOR framework, it investigates whether large-scale multimodal priors can enhance latent-space search and recovery. This approach aims to unify the generalization capabilities of foundation models with the adaptability of search-based control, addressing limitations that arise when either approach is used in isolation.

### 2.3 Literature Review Summary

This literature review examined how robotic control has progressed from imitation learning toward predictive, adaptive frameworks and large-scale foundation models. Behavioural cloning enables efficient learning from demonstrations but suffers from distribution shift and compounding errors during execution. Model-based approaches such as Dreamer introduced latent world models that enable agents to imagine and evaluate future trajectories, providing a basis for predictive control.

SAILOR extends latent imagination into test time by performing online search and correction in a learned latent space, enabling recovery from distribution shift without additional expert supervision. In parallel, VLA models such as $\pi_0$ provide strong semantic and visuomotor generalization through large-scale multimodal pretraining but remain feedforward and lack test-time reasoning or recovery mechanisms.

The literature therefore reveals a gap between adaptive, search-based control methods and highly generalizable foundation models. This thesis addresses that gap by evaluating whether a pretrained VLA model, $\pi_0$-Droid, can serve as an effective base policy within the SAILOR framework, combining large-scale generalization with latent-search-based recovery.

## 3 Reproducing the SAILOR Baseline

This chapter presents the reproduction of the SAILOR framework on RoboMimic manipulation tasks and establishes the baseline used for all subsequent modifications in this thesis.

### 3.1 Compute Infrastructure and Apptainer-Based Software Environment

All experiments were conducted on the Nibi cluster at Compute Canada using the SLURM workload manager. Training jobs were allocated a single Multi-Instance GPU (MIG) slice of an NVIDIA H100 (40 GB VRAM), corresponding to half of a full 80 GB H100 device, along with

one CPU core and approximately 16 to 18 GB of system memory. This configuration was sufficient to run the full SAILOR pipeline.

Due to strict versioning requirements across the SAILOR codebase, RoboMimic datasets, and their underlying dependencies, all experiments were executed inside a containerized environment using Apptainer, following Compute Canada's recommended workflow. Rather than relying on the host system's Python environment, a Conda environment was built and activated inside the Apptainer container, which allowed precise control over package versions required by SAILOR and RoboMimic.

Significant effort was required to resolve software compatibility issues arising from mismatches between system libraries, GPU drivers, and Python packages. In particular, the original PyTorch version specified by SAILOR was incompatible with the Compute Canada environment and was upgraded to a newer release to ensure correct CUDA support and stable execution on H100 GPUs. In addition, MuJoCo rendering was configured to use OSMesa explicitly, enabling headless simulation and avoiding graphics driver dependencies that are unsupported on compute nodes.

Once these issues were resolved, all preprocessing, training, and evaluation jobs were launched via SLURM batch scripts that mounted the project repository into the container and executed SAILOR commands inside the controlled Conda environment. This compute and software setup resulted in a stable, reproducible execution pipeline that matched Compute Canada best practices and was necessary before reliable experimental results could be obtained.

### 3.2 RoboMimic Dataset Preparation and Preprocessing

Experiments used expert demonstration datasets from the RoboMimic benchmark suite [3], which provides standardized manipulation tasks and evaluation protocols. For each task, raw demonstration datasets were downloaded using scripts provided in the SAILOR repository and stored under the expected directory structure.

RoboMimic datasets store demonstrations as state-based trajectories. To support image-based diffusion policies and SAILOR training, these datasets were converted into observation-based formats using the official RoboMimic preprocessing utilities. This preprocessing step renders RGB observations from two camera views, agentview and robot0 eye-in-hand, resizes images to 64×64

resolution, includes proprioceptive state information, and applies shaped rewards with episode termination on task success.

Preprocessing was performed inside the Apptainer container on the Nibi cluster to avoid discrepancies between local and cluster environments. Once resolved, preprocessing produced datasets fully compatible with the SAILOR training pipeline without requiring modifications to the original code.

### 3.3 Published SAILOR Baseline and Benchmark Scope

The first objective of this thesis was to reproduce the published SAILOR [4] baseline on standard RoboMimic benchmarks. This was necessary for three reasons. First, it validated that the original training and evaluation pipeline was correctly configured in the present compute environment. Second, it established a trusted reference point for all later comparisons. Third, it provided a practical understanding of the recovery loop, policy variants, and codebase before replacing the original diffusion-policy base with $\pi_0$-Droid.

The original SAILOR paper reports success rates on three RoboMimic manipulation tasks of increasing difficulty: Lift, Can, and Square. Lift requires grasping and lifting a block above a height threshold. Can is a pick-and-place task that requires grasping a cylindrical object and placing it into a bin. Square is a nut-assembly task that requires inserting a square object onto a matching peg and demands precise alignment and longer-horizon control.

Although SAILOR was evaluated more broadly across multiple benchmarks in the original paper, this thesis focuses on the RoboMimic portion of that setup because it provides a standardized and reproducible benchmark with fixed demonstration datasets and evaluation procedures. Within RoboMimic, the Lift, Can, and Square tasks were selected because they are highlighted in the original SAILOR results and provide a useful progression in difficulty, from short-horizon grasping to recovery-heavy long-horizon manipulation.

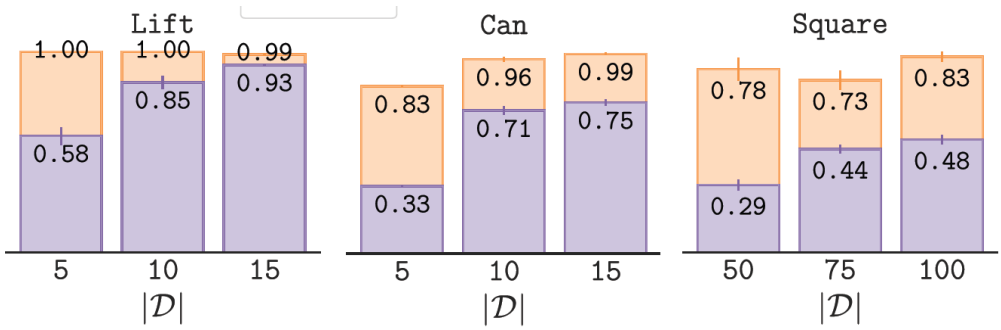


**Figure 3.1: Original SAILOR Paper Results Comparing Diffusion Policy (DP) and SAILOR on Three RoboMimic Tasks [4].**

Figure 3.1 shows the success rates reported in the original SAILOR paper. Each bar corresponds to a different number of expert demonstrations. Across all tasks, SAILOR substantially outperforms the diffusion-policy baseline, particularly in low-data regimes and on longer-horizon tasks such as Square. These results demonstrate the benefit of latent-space planning for recovery under distribution shift.

These published results serve as the reference point for the reproduction experiments described in Sections 3.4–3.7.

### 3.4 Baseline Training Pipeline

Using the official SAILOR codebase and default configuration files, the full baseline training pipeline was executed end-to-end on the Lift, Can, and Square tasks without modifying the underlying algorithm. Training was performed using the provided training command, beginning with diffusion-policy pretraining and continuing through SAILOR training and policy distillation. No algorithmic components or hyperparameters were modified.

Training in SAILOR is defined by a fixed environment-step budget rather than a fixed number of epochs. Following the original paper, approximately 100,000 environment steps were used for Lift, and approximately 500,000 steps were used for Can and Square.

Within this budget, training proceeds in multiple phases. First, a diffusion-based visuomotor policy is pretrained using behavioral cloning on expert demonstrations. In the default configuration, this pretraining stage consumes roughly 20 percent of the total environment-step budget. During this stage, the policy learns to imitate expert behavior but does not perform any planning or recovery.

After diffusion-policy pretraining, SAILOR initializes its internal world model, reward model, and critic. During an initial warm-start phase, the pretrained diffusion policy is executed in the environment with injected noise to collect rollouts. These rollouts, together with expert demonstrations, are used to train the latent world model and evaluation networks. At this stage, SAILOR does not yet perform latent-space planning.

Once initialized, SAILOR enters its online learning-to-search phase. Training proceeds in repeated cycles of environment interaction, latent-space planning, and model updates. During execution, SAILOR performs search in its learned latent space to generate corrective action sequences. Candidate trajectories are evaluated using the learned reward and critic models, and the first action of the highest-scoring sequence is executed before replanning. Over time, high-quality trajectories discovered through search are distilled back into the diffusion policy, producing a distilled policy that can be executed without test-time planning.

### 3.5 Evaluation Protocol and Policy Variants

For this baseline-reproduction milestone, performance was measured primarily using success rate, defined as the percentage of evaluation episodes that satisfy the environment-specific success condition. This follows the original SAILOR evaluation setup and allows direct comparison to the published baseline results.

Evaluation is performed by generating evaluation videos, each of which contains ten parallel environment rollouts executed under the same policy. The success rate associated with a video is computed as the fraction of successful rollouts within that batch and is encoded directly in the output filenames produced by the SAILOR evaluation scripts. For example, a filename containing "succ_0.80" indicates that 8 out of 10 rollouts were successful.

Diffusion-policy evaluations are performed at the end of diffusion-policy pretraining, reflecting performance under pure behavioral cloning. During SAILOR training, evaluation videos are generated periodically throughout training and at the end of the allocated environment-step budget, reflecting performance of the full SAILOR controller with latent-space search enabled. For the distilled diffusion policy, evaluation is performed after distillation updates, measuring the performance of the policy without test-time planning.

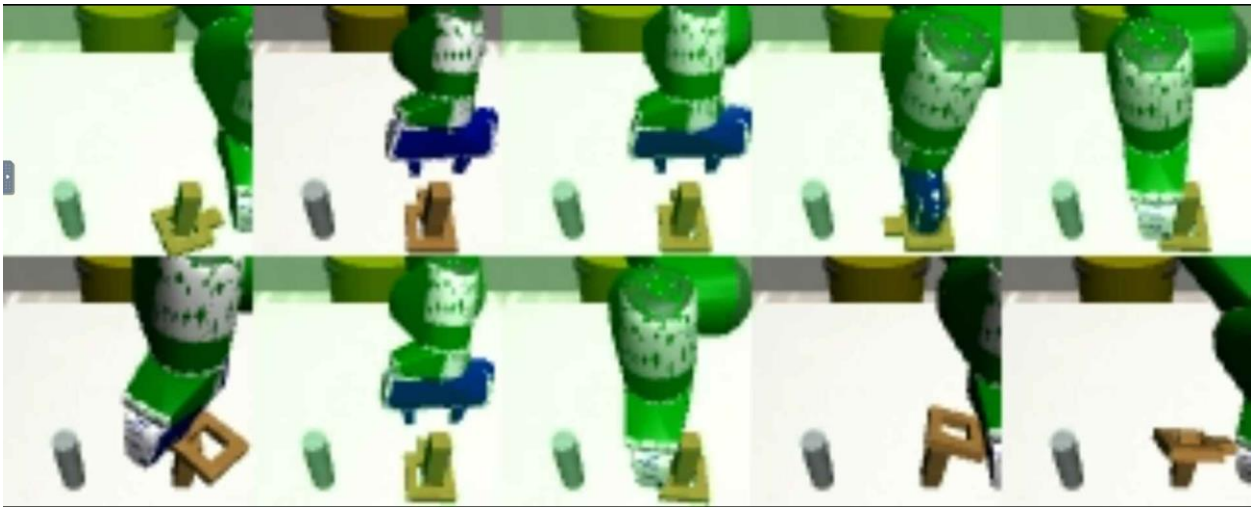

**Figure 3.2: Representative Evaluation Frame from the Square Task Illustrating Success Indicators Across Parallel Rollouts.**

Figure 3.2 shows a representative evaluation frame from the Square task. Each panel corresponds to a parallel environment rollout at the end of an episode, with success indicated by the environment's built-in success signal. These indicators form the basis for the per-video success rates encoded in evaluation filenames.

Accordingly, three policy variants were evaluated in this baseline study:

1. Diffusion Policy (DP) trained only via behavioral cloning.
2. DP-Distilled, obtained after distillation from SAILOR's search-corrected trajectories.
3. SAILOR (Online), which performs latent-space search at test time.

### 3.6 Baseline Results

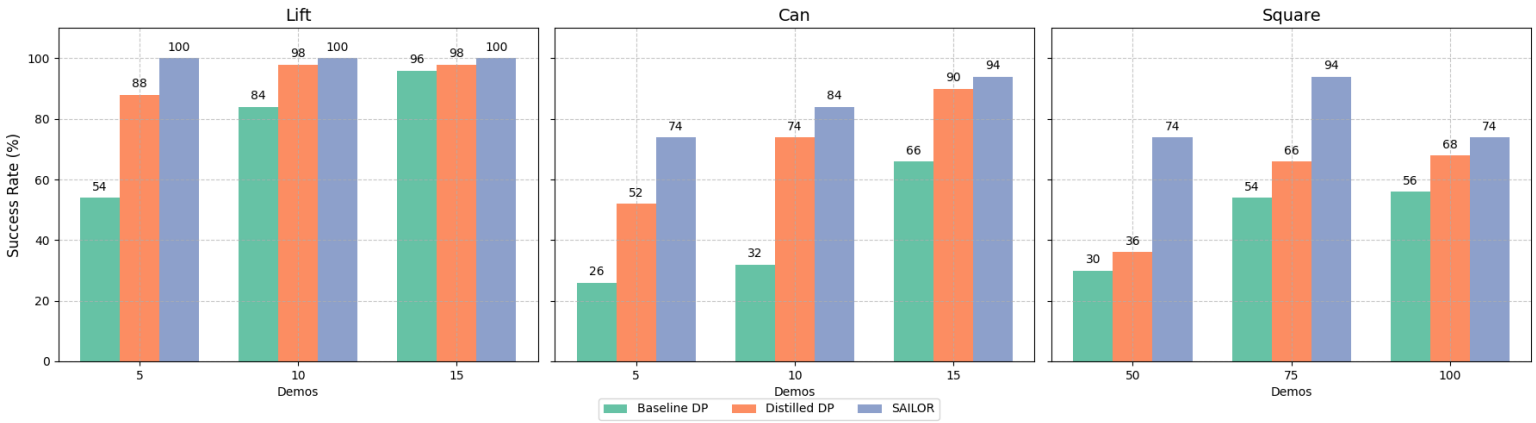


**Figure 3.3: Final Success Rates at the End of Training for the Diffusion-Policy Baseline, Distilled Policy, and Full SAILOR Controller Across All Three Tasks.**

Figure 3.3 summarizes the final reproduced success rates obtained at the end of training for each policy variant and task.

The reproduced success rates for the diffusion-policy baseline and the full SAILOR controller closely match those reported in the original SAILOR paper, with consistent relative improvements across tasks. Minor numerical differences are expected, as the original results are averaged over three random seeds, whereas the experiments reported here use a single seed for each configuration.

Across all tasks, enabling SAILOR's latent-space search improves success relative to the pure diffusion-policy baseline by helping correct small execution errors that would otherwise accumulate under distribution shift. These gains are most pronounced in low-data regimes, where diffusion policies trained purely via imitation struggle to recover once they deviate from expert trajectories.

Distillation consistently improves performance over the baseline diffusion policy. The distilled policy captures a significant portion of SAILOR's corrective behavior while remaining a purely reactive controller. However, for more challenging tasks such as Square, the distilled policy does not fully match the performance of online SAILOR.

### 3.7 Interpretation and Takeaways

Several consistent trends emerge from the reproduced baseline results. Latent-space search substantially improves performance across all tasks, particularly when demonstration data is limited. Diffusion policies trained purely via imitation are more susceptible to error under distribution shift, while search-based execution enables recovery from small execution errors.

Policy distillation captures many of these improvements, indicating that a large fraction of search-discovered corrections can be compiled into a static controller; however, some recovery behaviours, especially those involving precise alignment or longer horizons, continue to benefit from test-time planning.

Task difficulty plays a clear role. Lift approaches saturation quickly even under imitation learning. Can and Square benefit more strongly from latent-space search, with Square remaining the most challenging task due to its sensitivity to compounding errors.

### 3.8 Chapter Summary

This chapter established a validated SAILOR baseline on RoboMimic. The reproduced results matched the main trend reported in the original paper: online SAILOR outperformed the pure diffusion-policy baseline, distillation captured part of the improvement, and task difficulty increased from Lift to Can to Square. These results provide the reference system against which the later $\pi_0$-based stages of the thesis are evaluated.

## 4 Enabling $\pi_0$-Droid Inference in RoboMimic

### 4.1 Milestone Objective

The second milestone of this thesis was to make the pretrained $\pi_0$-Droid policy run correctly inside the RoboMimic simulation environment used by the reproduced SAILOR baseline. This stage followed baseline reproduction and preceded task-specific fine-tuning and full integration into SAILOR. Its purpose was not yet to demonstrate strong benchmark performance, but to establish a correct and reproducible inference interface between $\pi_0$-Droid and RoboMimic.

This turned out to be an interface-alignment problem rather than a simple model swap. In the reproduced RoboMimic setup from Chapter 3, the original diffusion-policy baseline operated under RoboMimic's default 7-dimensional end-effector action interface, consisting of a 6-DoF end-effector delta pose and a gripper command [4]. The diffusion policy in the reproduced setup

also generated multi-step action chunks, but those chunks were expressed in RoboMimic's default task-space format. By contrast, $\pi_0$-Droid returned 10-step action chunks in an 8-dimensional joint-space format consisting of seven joint-velocity commands and one gripper command [30], [33]. Before $\pi_0$-Droid could be evaluated fairly, the simulator therefore had to be changed to match the policy's native control and observation contract.

The practical goals of this milestone were therefore to:

1. build a runtime environment that could execute OpenPI and RoboMimic together on Compute Canada;
2. map RoboMimic observations into the DROID-style input format used by $\pi_0$-Droid;
3. switch simulator execution from the default task-space controller to joint-velocity control; and
4. correct the gripper-state and gripper-command conventions.

The result sought in this chapter was a working inference path. In other words, $\pi_0$-Droid needed to load, accept simulator observations in the correct format, and produce executable actions under a consistent simulator controller.

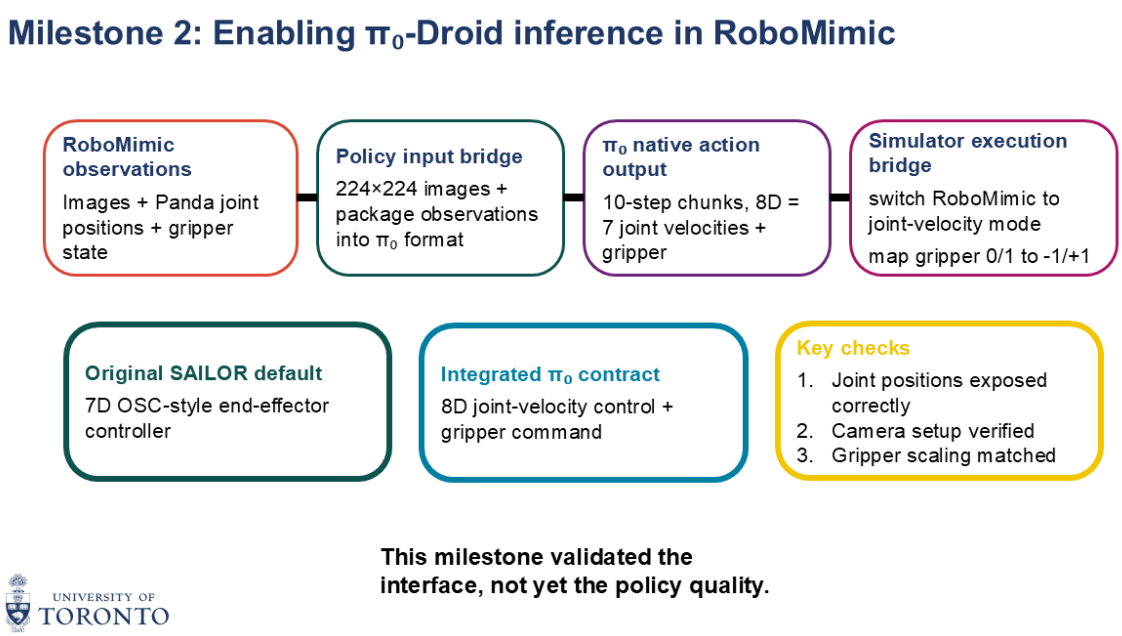


**Figure 4.1: Milestone-2 interface alignment for enabling $\pi_0$-Droid inference in RoboMimic.**

Figure 4.1 summarizes the observation bridge from RoboMimic to $\pi_0$-Droid, the 224×224 policy input formatting, the model's native 10-step 8-dimensional joint-velocity output, and the simulator-side execution bridge required to replace the default task-space controller and correct gripper semantics.

### 4.2 Rebuilding the Runtime on Compute Canada

Before any simulator experiments could be run, a unified runtime environment had to be established on the Compute Canada cluster. Chapter 3 had already required an Apptainer-based workflow with a controlled Conda environment to reproduce SAILOR reliably. Milestone 2 extended that environment to support OpenPI, JAX, and the DROID inference stack without breaking the previously working RoboMimic setup.

This step repeated the same general class of software problems already encountered in Section 3.1, but with a different set of packages. Significant effort was again required to resolve compatibility issues arising from mismatches between system libraries, GPU drivers, and Python packages. In this stage, the most persistent problems were computer-vision import errors, shared-library conflicts, and unwritable cache and temporary directories inside the container. The main blocking issue turned out not to be a fundamental glibc incompatibility, but rather a combination of OpenCV and image-library ABI mismatches together with runtime cache locations that were not writable on compute nodes.

The final working setup rebuilt the Conda environment inside Apptainer so that it preserved the original SAILOR dependencies while also supporting OpenPI. In practice, conflicting image-related packages first had to be removed from the Conda environment, and the working combination was then restored using pip-installed packages together with supporting system libraries such as OpenCV, libtiff, JPEG, and GLib. The runtime also redirected package caches and temporary directories for components such as Matplotlib, fontconfig, and Numba into writable scratch locations, and it was necessary to force the environment to prefer the Conda-managed shared libraries through LD_LIBRARY_PATH and LD_PRELOAD [33], [36].

A useful methodological lesson also came from this step. The initial plan was to mimic the OpenPI notebook-style examples as closely as possible by trying to run inference through JupyterLab inside the Apptainer Conda environment. In practice, that approach proved unreliable on Nibi and

consumed time without producing a stable workflow. Progress accelerated once the milestone switched to explicit standalone scripts that performed imports, loaded checkpoints, ran inference, and printed structured diagnostics. That script-based path became the reproducible foundation for the rest of the milestone.

### 4.3 Standalone $\pi_0$-Droid Validation and Scope of the Model Choice

The first reliable proof that $\pi_0$-Droid could run in the target environment came from the standalone script `pi0_trial.py` [33]. That script loaded the `pi0_droid` configuration, downloaded the official checkpoint, created a trained policy object, constructed a DROID-style example input, and executed `policy.infer(example)` successfully.

This standalone validation established three important facts. First, OpenPI inference itself was functional on the cluster. Second, the model input schema included an exterior image, a wrist image, joint position, gripper position, and a prompt [30], [33]. Third, the returned action tensor had shape

$$(10\,,8),$$

confirming that $\pi_0$-Droid produced a 10-step chunk of 8-dimensional actions [33].

During this stage, several OpenPI model variants were also explored briefly, including $\pi_0$-Droid, $\pi_{0.5}$-Droid, and $\pi_0$-FAST-DROID, as listed in the OpenPI repository [29]. These experiments were useful for understanding the available checkpoints, but they did not materially improve zero-shot performance and were not allowed to dominate the project. The focus of the thesis was not to benchmark the full OpenPI model family, but to study whether a strong pretrained VLA prior could be integrated into SAILOR. For that reason, $\pi_0$-Droid remained the primary model for the thesis because it best matched the DROID and Franka control setting relevant to RoboMimic.

This stage also clarified the intended robot embodiment behind the DROID checkpoint. The relevant DROID setup is Franka-based, not UR5-based. The DROID materials describe a Franka Panda tabletop manipulation platform with a gripper and multi-camera observations [1], [31], [32]. Since RoboMimic tasks are also built around Franka Panda manipulation [3], the main mismatch here was not robot identity but control representation.

### 4.4 Constructing the Observation Bridge

Once standalone inference had been validated, the next step was to make RoboMimic observations compatible with the input schema expected by $\pi_0$-Droid. The working DROID-style example used by the model contained five fields: a single exterior RGB image, a wrist RGB image, joint position, gripper position, and a prompt [30], [33].

The final simulator-side bridge therefore performed four observation operations:

1. map `agentview_image` to the single exterior-camera input;
2. map `robot0_eye_in_hand_image` to the wrist-camera input;
3. extract a 7-dimensional Panda joint-position vector directly from simulator `qpos`; and
4. convert the simulator gripper state into the scalar format expected by $\pi_0$-Droid [34], [36].

A major correction in this part of the milestone concerned image preprocessing. Early versions of the runtime effectively upscaled simulator images without ensuring that the actual input path matched the DROID-style preprocessing contract. This was later replaced by an explicit 224×224 image path with aspect-ratio-preserving resize and padding. In the final runtime, image size, reward shaping, done mode, and task time limits were owned directly by the $\pi_0$ evaluation script rather than inherited implicitly from older SAILOR defaults [36]. This change mattered later as well, because the same 224×224 visual path was retained in the fine-tuning stage.

The final observation bridge was therefore not just a convenience wrapper. It defined the exact image, proprioceptive, and prompt inputs that would also be reused in the following milestones.

### 4.5 Camera Settings Available in Simulation

Another issue investigated in this milestone was whether the simulator needed to reproduce specific camera intrinsics from DROID data collection. Since $\pi_0$-Droid was trained on real robot data, it was natural to ask whether simulation should match focal length, sensor size, aperture, or a fixed intrinsic matrix.

The conclusion was that this was neither possible nor necessary. DROID data were collected across many real camera calibrations rather than one fixed canonical camera configuration [1], [31], [32]. The OpenPI inference path consumed resized RGB images rather than explicit camera intrinsics

[30]. On the simulator side, RoboMimic and robosuite exposed camera pose, field of view, and image resolution, but not physical focal length, sensor size, or aperture [39].

As a result, this milestone did not attempt to reproduce a single physical DROID camera specification. The practical camera decisions were instead to use reasonable viewpoints, adequate render quality, and the correct 224×224 preprocessing path.

### 4.6 Switching the Simulator to Joint-Velocity Control

The most important change in milestone 2 was the controller. A direct substitution of $\pi_0$-Droid into the reproduced baseline would have been invalid, because the reproduced RoboMimic diffusion-policy baseline expected the default 7-dimensional task-space action

$$a^{\text{RoboMimic}} = (\Delta x, \Delta y, \Delta z, \Delta r_x, \Delta r_y, \Delta r_z, g),$$

where the first six dimensions specify end-effector motion and the final dimension controls the gripper [4]. By contrast, $\pi_0$-Droid returned 8-dimensional joint-space actions

$$a^{\pi_0} = (\dot{q}_1, \dot{q}_2, \dot{q}_3, \dot{q}_4, \dot{q}_5, \dot{q}_6, \dot{q}_7, g),$$

with seven joint velocities and one gripper command [30], [33].

So the core mismatch was not simply that one policy predicted chunks and the other did not. The original diffusion-policy baseline also predicted multi-step plans. The real mismatch was that the reproduced RoboMimic baseline used a 7-dimensional task-space action representation, whereas $\pi_0$-Droid used an 8-dimensional joint-space action representation.

An early possibility was to preserve the existing controller and convert $\pi_0$-Droid's joint-space outputs back into task-space commands through an additional kinematic layer. That approach was rejected because it would have introduced an extra interpretation layer between the learned policy output and the simulator. The cleaner solution was to change the simulator rather than reinterpret the policy.

The final runtime therefore switched RoboMimic to `JOINT_VELOCITY` control and enforced that contract explicitly [34], [37]. This was the key step that made simulator execution consistent with the policy's native output. It also became important later, because the same joint-velocity action mapping was reused in the fine-tuning and integration milestones.

### 4.7 Gripper Observation and Execution Mapping

The gripper path required two separate corrections: one on the input side and one on the output side.

#### 4.7.1 Gripper Observation Normalization

For policy input, $\pi_0$-Droid expects observation/gripper_position in the normalized range $[0, 1]$, where 0 means open and 1 means closed [30]. The simulator, however, exposed Panda finger-joint positions directly in qpos. Let $q_{f1}$ and $q_{f2}$ denote the two Panda finger-joint positions. The normalized gripper input used by the runtime was

$$g_{\text{obs}} = 1 - \text{clip}\left(\frac{\max(|q_{f1}|, |q_{f2}|)}{0.04}, 0, 1\right).$$

Under this convention, a fully open Panda gripper maps to $g_{\text{obs}} = 0$, and a fully closed gripper maps to $g_{\text{obs}} = 1$ [35].

A further issue emerged at reset time. The real DROID runtime assumes the gripper starts fully open, but the robosuite Panda gripper often initialized to a half-open state. Without correction, the first gripper observation given to the model was already off-distribution. The final runtime therefore forced the Panda gripper to a fully open state after every reset [35].

#### 4.7.2 Gripper Execution Mapping

For policy output, the issue was different. After inference, the DROID-style path treated the last action dimension as a thresholded binary gripper signal [30]. RoboMimic and robosuite, however, execute gripper commands in the range $[-1, +1]$, with negative values corresponding to open and positive values to close [38].

Let $a_g$ denote the final gripper output from the model. The simulator-side mapping used by the runtime was

$$g_{\text{bin}} = \mathbf{1}[a_g > 0.5], \qquad g_{\text{exec}} = 2g_{\text{bin}} - 1.$$

This maps model output 0 to simulator command $-1$ and model output 1 to simulator command $+1$. The final action batch sent to the simulator therefore remained consistent with the gripper convention expected by RoboMimic and robosuite [35], [38].

These gripper corrections were crucial. They were also discovered gradually. Early debugging focused on arm-control dimensionality, but later experiments showed that gripper-state scaling, reset initialization, and execution sign conventions had a large effect on behavior.

### 4.8 Prompts, Chunk Execution, and Evaluation Conditions

Because $\pi_0$-Droid is a vision–language–action policy, each rollout required a prompt. To keep this stage controlled and benchmark-aligned, the prompts were chosen directly from the RoboMimic task goals used in the SAILOR benchmark:

- Lift: "lift the block"
- Can: "place the can in the correct bin"
- Square: "insert the square tool into the slot" [4].

A more descriptive Lift prompt was also tried:

> "Slowly Align yourself and gripper with block then lift it above the desk."

In practice, that more descriptive prompt did not improve zero-shot behavior and performed worse than the shorter benchmark-style prompt. For that reason, the final prompt set remained simple and task-directed.

A second practical decision concerned chunked execution. The standalone validation showed that $\pi_0$-Droid produced action chunks of length 10 [33]. In the final runtime, the policy was queried for a chunk and then run open-loop for 8 actions before requesting a new chunk, matching the practical guidance in the DROID runtime example that 8 steps is a reasonable default [30]. That same open-loop setting was kept for the remainder of the project.

Milestone 2 used the same three RoboMimic tasks as Chapter 3: Lift, Can, and Square [4]. The same per-episode time limits were also retained. Lift used a 100-step limit, while Can and Square used 200-step limits [4]. In the environment metadata inspected during this project, `control_freq = 20`, so one `env.step(action)` corresponded to a 0.05 s control interval. Therefore,

$$100 \text{ steps} = 5.0 \text{ s}, \qquad 200 \text{ steps} = 10.0 \text{ s}.$$

These episode limits were kept fixed throughout milestone 2 and throughout the later $\pi_0$-based stages of the thesis. This mattered, because those time limits affected the results and needed to remain consistent across stages [39].

### 4.9 Zero-Shot Results

Once the observation bridge, controller switch, gripper corrections, prompts, and evaluation limits were in place, $\pi_0$-Droid was evaluated in zero-shot form on Lift, Can, and Square. In these initial experiments, the pretrained base $\pi_0$-Droid policy failed on all three tasks.

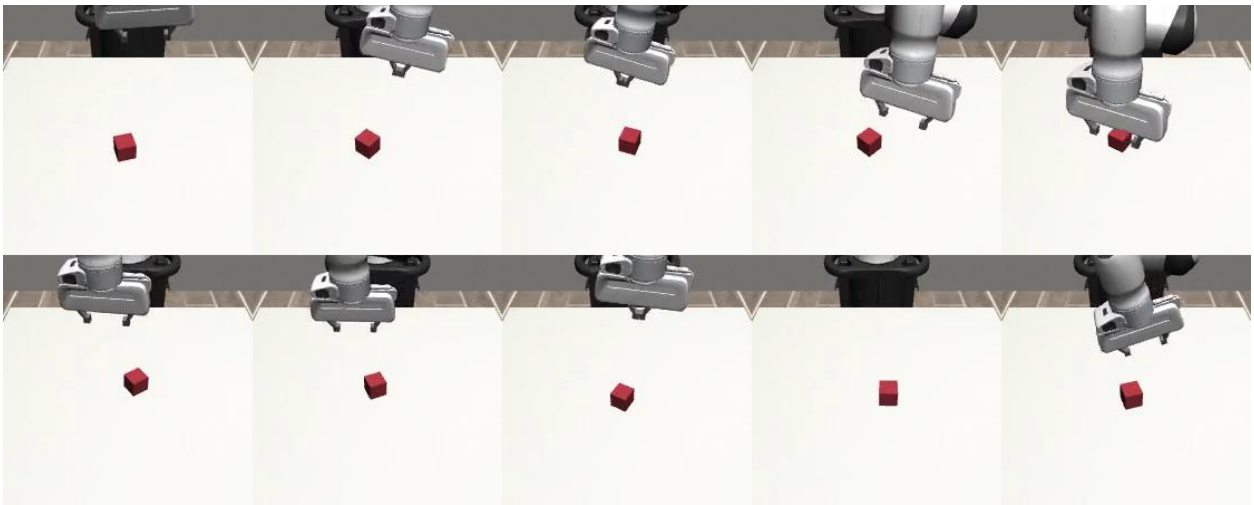

**Figure 4.2: Representative final frame from zero-shot $\pi_0$-Droid evaluation on the Lift task before fine-tuning.**

Figure 4.2 illustrates zero-shot failure under the configured observation and controller interface. At this stage, the runtime was executing end-to-end, but task success had not yet been achieved.

This result was interpreted cautiously. At this stage, poor zero-shot performance did not by itself imply that the interface was wrong. The policy had been trained on real-world DROID data and was being evaluated here from simulated camera observations without any task-specific adaptation [1], [31], [32]. The interface was only really validated later, when the same action mapping and observation path produced improved behavior after fine-tuning.

### 4.10 Chapter Summary

This milestone established a working $\pi_0$-Droid inference path inside RoboMimic. The key result was a reproducible runtime in which RoboMimic observations were converted into the DROID-

style policy input format, the simulator executed 8-dimensional joint-velocity actions instead of the default 7-dimensional task-space actions, and the gripper observation and execution semantics were corrected.

This stage was essential because the same observation mapping, joint-velocity action mapping, and controller choice were carried forward into the fine-tuning and integration milestones. Without milestone 2, later results would have remained ambiguous, since any improvement or failure could still have been attributed to an interface bug rather than to the policy itself.

# 5 Task-Aligning $\pi_0$-Droid through JAX LoRA Fine-Tuning

## 5.1 Milestone Objective

The third milestone of this thesis was to task-align the pretrained $\pi_0$-Droid policy to the RoboMimic benchmark before integrating it into SAILOR. Chapter 4 established that $\pi_0$-Droid could run end-to-end inside RoboMimic under a corrected observation and action interface, but it also showed that zero-shot execution was insufficient on Lift, Can, and Square. The next step was therefore to adapt the pretrained VLA to the benchmark task distribution through offline imitation learning, before adding any search or recovery mechanism.

This stage was also necessary for fairness. In the original SAILOR pipeline, the diffusion-policy base is already task-aligned before latent-space search is added [4]. Replacing that base with a completely zero-shot $\pi_0$-Droid policy would therefore have created an unfair comparison between a task-trained diffusion baseline and an out-of-domain VLA prior. Milestone 3 was designed to correct that asymmetry by adapting $\pi_0$-Droid to the same RoboMimic task family before later integration.

Accordingly, this milestone had two purposes. First, it established whether $\pi_0$-Droid could be adapted effectively to RoboMimic through parameter-efficient fine-tuning while preserving the validated interface from Chapter 4. Second, it built the fine-tuning path that would later be needed in Milestone 4, where SAILOR's corrected trajectories would have to be distilled back into the base policy.

## 5.2 Choosing JAX LoRA Instead of PyTorch Full Fine-Tuning

The initial plan for this milestone was to convert the released JAX $\pi_0$-Droid checkpoint to PyTorch and then fine-tune the converted model before integration into the PyTorch-based SAILOR

codebase. In principle, that path seemed attractive because it would reduce framework boundaries later in the project. In practice, it was not the most suitable route.

The main reason was support and cost. OpenPI's PyTorch implementation supported fine-tuning, but it did not support LoRA training at the time of this project [29]. This meant that the practical PyTorch training path would have been full fine-tuning rather than parameter-efficient adaptation. OpenPI's own memory guidance estimated more than 22.5 GB for LoRA fine-tuning and more than 70 GB for full fine-tuning on a single GPU [29], making full PyTorch fine-tuning a much more expensive option. By contrast, the repository already provided a supported JAX LoRA path. Since the goal of the thesis was to test whether a pretrained VLA could work inside SAILOR, the cleaner choice was to keep $\pi_0$-Droid in JAX for training, use LoRA for parameter-efficient adaptation, and defer framework-bridging concerns to the later integration milestone.

This decision kept the milestone focused. Rather than spending time forcing an unsupported or unnecessarily expensive training path, the project used the route that fit both the repository support and the thesis scope.

### 5.3 Building a RoboMimic HDF5 Fine-Tuning Pipeline for OpenPI

A key design decision in this milestone was to keep the fine-tuning data path consistent with the validated interface from Chapter 4. OpenPI's reference training workflow is organized around its own dataset conventions [29], but the rest of this thesis already relied on RoboMimic HDF5 data and the original SAILOR pipeline. To preserve consistency, a custom offline bridge was built from RoboMimic HDF5 directly into the OpenPI JAX training path instead of moving through a more distant dataset conversion pipeline.

This local training bridge consisted of four main components. The dataset adapter `openpi_robomimic_hdf5_dataset.py` emitted RoboMimic samples in the DROID-style format expected by $\pi_0$-Droid [43]. The local data configuration file `openpi_robomimic_data_config.py` connected that dataset to OpenPI's standard DROID transform and normalization path [42]. The training entry point `train_pi0_droid_lora_robomimic.py` kept the upstream JAX training structure while replacing only the data source [41]. Finally, the sanity-check script `sanity_pi0_robomimic_loader.py` was used to verify the raw and transformed data contracts before launching long training runs [40].

This design was deliberate. The custom path changed only what was necessary to feed RoboMimic HDF5 into OpenPI. The model, optimizer, checkpoint loading, and training logic remained close to the supported upstream path [29], [41]. It also allowed the milestone to stay close to the same benchmark data family that would later be used again during SAILOR integration.

The fine-tuning path also reused the same observation and action contract established in Chapter 4. Each sample used a single external RGB image, a wrist RGB image, Panda joint positions, normalized gripper position, and a task prompt as inputs, together with the same 10-step, 8-dimensional joint-velocity-plus-gripper action target used during runtime inference [34]–[36], [42], [43]. The same 224×224 visual preprocessing path was preserved for consistency across fine-tuning, evaluation, and later integration [36], [42], [43].

### 5.4 JAX LoRA Training Implementation

The fine-tuning entry point for this milestone was train_pi0_droid_lora_robomimic.py [41]. Its design goal was to remain as close as possible to OpenPI's upstream training path while replacing only the dataset layer. Rather than rewriting the model, optimizer, checkpointing, or train-step logic, the implementation preserved the standard OpenPI JAX training structure and injected a local RoboMimic HDF5 dataset through the OpenPI transform and dataloader path [29], [41]–[43].

The training configuration followed the OpenPI LoRA pattern. The model was instantiated with paligemma_variant="gemma_2b_lora" and action_expert_variant="gemma_300m_lora", pretrained weights were loaded from the official pi0_droid checkpoint, the LoRA freeze filter was taken from the model configuration, and EMA was disabled for fine-tuning [29], [41]. The model retained the native pi0_droid action dimension rather than collapsing the network down to the raw 8-dimensional robot action width. This preserved compatibility with the standard OpenPI transform and checkpoint path while still training on RoboMimic-derived supervision.

The trainer also preserved OpenPI's checkpoint semantics so that later evaluation and conversion tools could consume the outputs without special-case handling. An important implementation detail was that the chosen training subset was resolved once and persisted to selected_demos.json, ensuring that resumed runs reused the same demonstration split rather than silently sampling a different subset [41]. This detail later mattered again in the SAILOR integration milestone, where

consistency of task-specific subsets became important for interpreting training and recovery behavior.

Before launching full training runs, the custom loader path was checked explicitly with sanity_pi0_robomimic_loader.py [40]. This script separated validation of the raw dataset contract from validation of the transformed OpenPI contract, making it easier to determine whether an error came from HDF5 extraction or from the later OpenPI transform stack. During optimization, only the LoRA-trainable subset of parameters was updated, while the pretrained backbone remained frozen in the usual parameter-efficient fine-tuning style [29], [41].

### 5.5 Training Setup and Benchmark Consistency

The fine-tuning runs were launched on Compute Canada using the same Apptainer-based workflow developed in the earlier milestones. The initial 24,000-step Lift run with 5 demonstrations used a single H100 40 GB MIG slice, one CPU core, and 32 GB of system memory. Inside the container, the run activated the robo_pi0 Conda environment, redirected caches into writable locations, disabled Weights & Biases logging, and set XLA_PYTHON_CLIENT_MEM_FRACTION=0.90 so that JAX could use most of the available GPU memory [41], [44], [46].

The launch scripts also defined the core fine-tuning configuration used throughout the milestone: task-specific offline demonstrations, action horizon 10, batch size 8, and a main budget of 24,000 training steps [41], [46]. This 24,000-step setting was chosen because it served the same task-alignment role that diffusion-policy pretraining serves in the original SAILOR pipeline, making the resulting base-policy comparison more meaningful [4]. The purpose of this milestone was not to maximize standalone $\pi_0$-Droid performance at any cost, but to create a fairer task-aligned base policy before adding search.

Fine-tuning was performed separately on Lift, Can, and Square using the same RoboMimic task family used throughout the thesis [3], [4]. For Lift and Can, the main fine-tuning comparisons used 5, 10, and 15 demonstrations. For Square, which was already known from the earlier milestones to be substantially harder and more data-hungry, the main comparisons used 50, 75, and 100 demonstrations. All evaluations used the same fixed benchmark horizons established in Chapters 3 and 4: 100 steps for Lift and 200 steps for Can and Square [4], [39]. Keeping these limits fixed

was important because they directly affected whether partial progress could be converted into binary task success [39].

After the main 24,000-step runs were completed, a second sweep resumed selected jobs to 48,000 total steps through the generic resume launcher and sweep scripts [44], [45]. These longer runs were used as follow-up analysis rather than as the primary milestone setting. This distinction matters in interpreting the results: the 24,000-step budget is the fairest main comparison to the original SAILOR base-policy stage, while the 48,000-step runs were used to test whether longer fine-tuning alone could close the remaining gap on harder tasks such as Square.

### 5.6 Main Fine-Tuning Results at 24,000 Steps

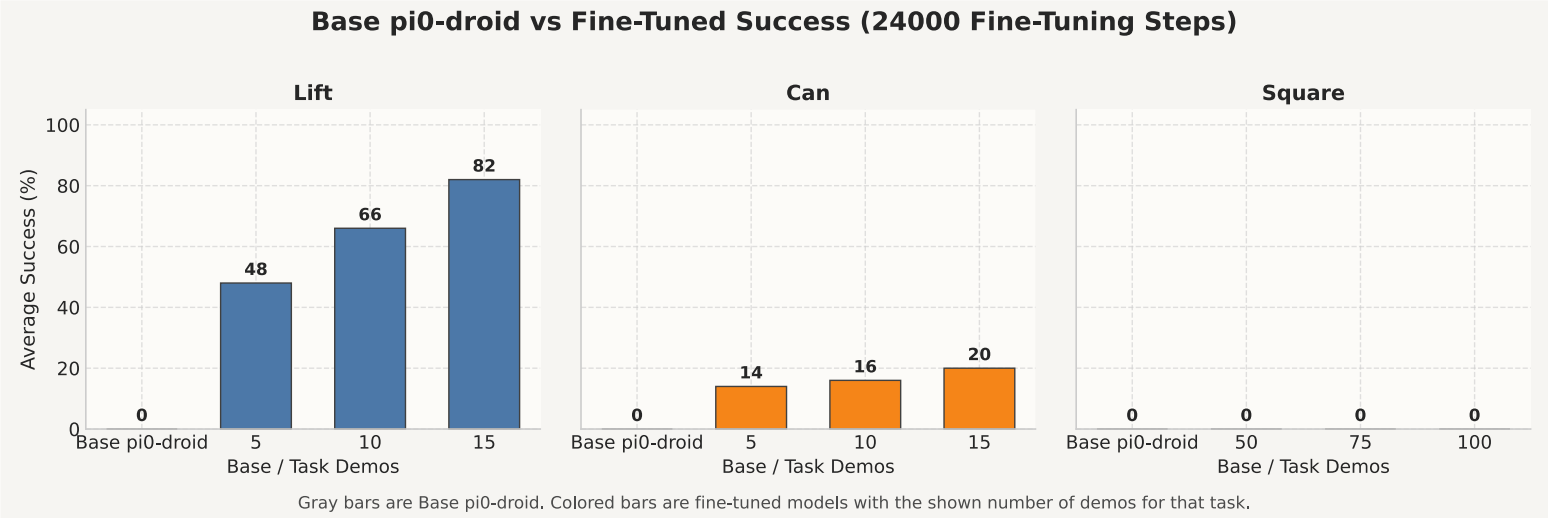


**Figure 5.1: Success rate for base $\pi_0$-Droid and JAX LoRA fine-tuned $\pi_0$-Droid after 24,000 fine-tuning steps on Lift, Can, and Square.**

Figure 5.1 shows the clearest result of this milestone. Fine-tuning substantially improved task success on Lift and produced smaller but still consistent gains on Can. The strongest success gains were observed on Lift, where performance rose from zero-shot failure to a clearly usable task policy as the number of demonstrations increased. Can followed the same overall trend, but at a lower performance level. Square remained at zero binary success across all tested demonstration counts.

This figure establishes the main result of Milestone 3: $\pi_0$-Droid could be task-aligned into a substantially stronger base policy on Lift and Can, even before any SAILOR search was added. At the same time, it also reveals the main limitation of fine-tuning alone. On the most precise and

time-sensitive task, better nominal behavior was still not enough to cross the benchmark's success threshold.

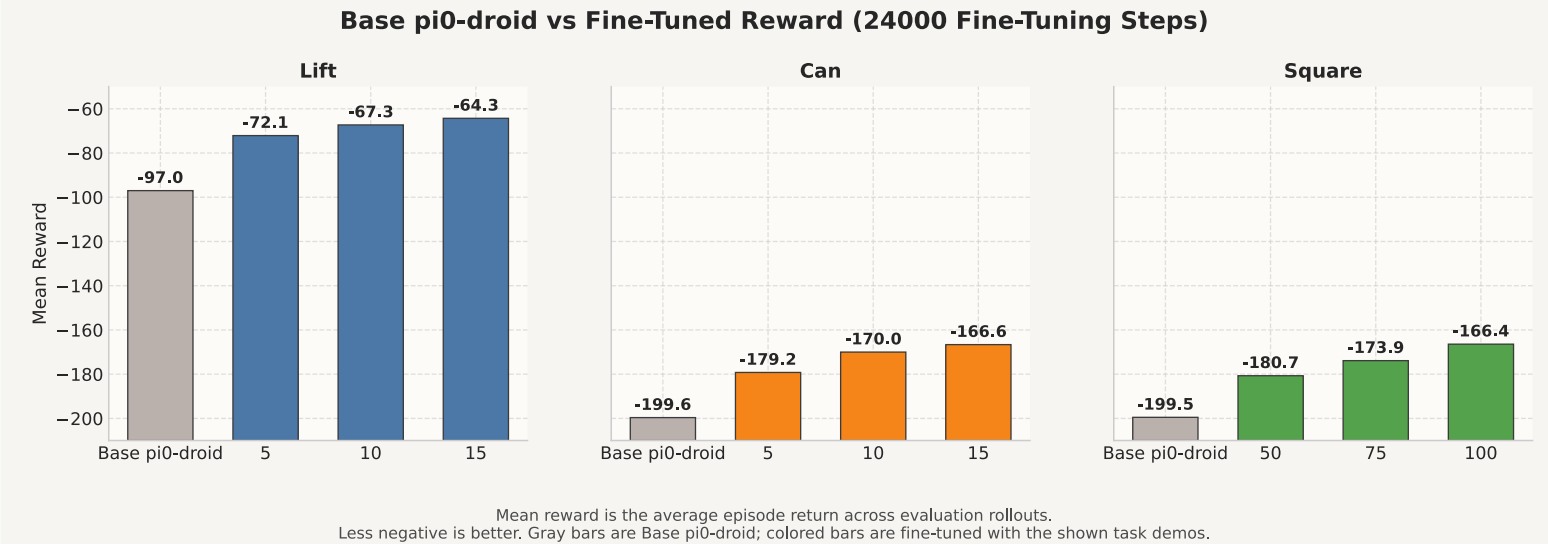


**Figure 5.2: Mean reward for base $\pi_0$-Droid and JAX LoRA fine-tuned $\pi_0$-Droid after 24,000 fine-tuning steps. Less negative is better.**

Figure 5.2 adds the necessary second metric. For Lift and Can, the reward trends reinforce the success trends: the fine-tuned policies are not only completing more episodes, but also achieving better overall task progress. The more important case is Square. Although binary success remains at zero, the reward improves substantially relative to the zero-shot base policy, with the best 24,000-step Square run improving from about -199.5 to about -166.4.

That distinction matters. The Square policy was not stagnant after fine-tuning. It was making more meaningful task progress, interacting with the object more effectively, and moving closer to the goal state, but it still could not finish the final insertion within the fixed evaluation horizon. Taken together, Figures 5.1 and 5.2 show that LoRA fine-tuning clearly improved task alignment, but that stronger pretrained priors alone were still not sufficient to solve the hardest task under the same benchmark limits.

### 5.7 Extended Fine-Tuning to 48,000 Steps

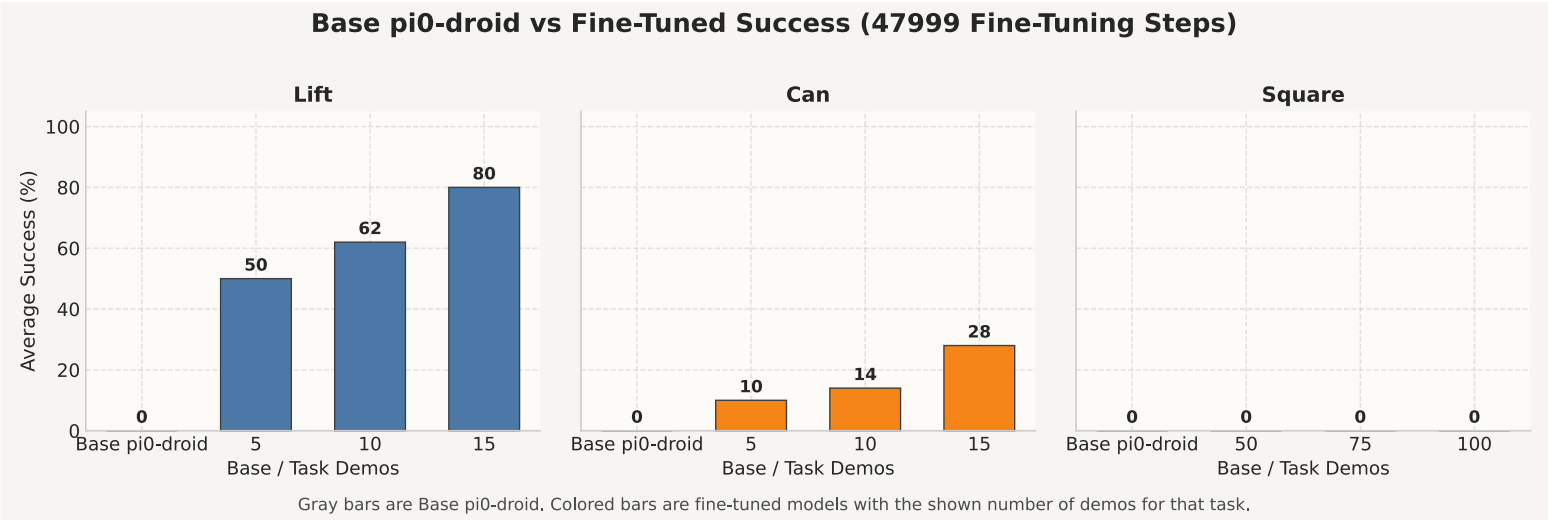


**Figure 5.3: Success rate after extending fine-tuning to 48,000 steps.**

Because the 24,000-step results suggested meaningful progress without full completion on the hardest task, selected runs were resumed to 48,000 total steps (47,999 in the final recorded checkpoint names). Figure 5.3 shows that the longer run did not change the main story on Lift, where performance was already strong after 24,000 steps. Can benefited somewhat more, especially at the highest demonstration count, where success rose to 28%. Square, however, still remained at zero success.

This follow-up showed that the remaining problem on Square was not simply a matter of stopping fine-tuning too early. More training could help some settings, but it did not remove the central completion gap on the hardest task.

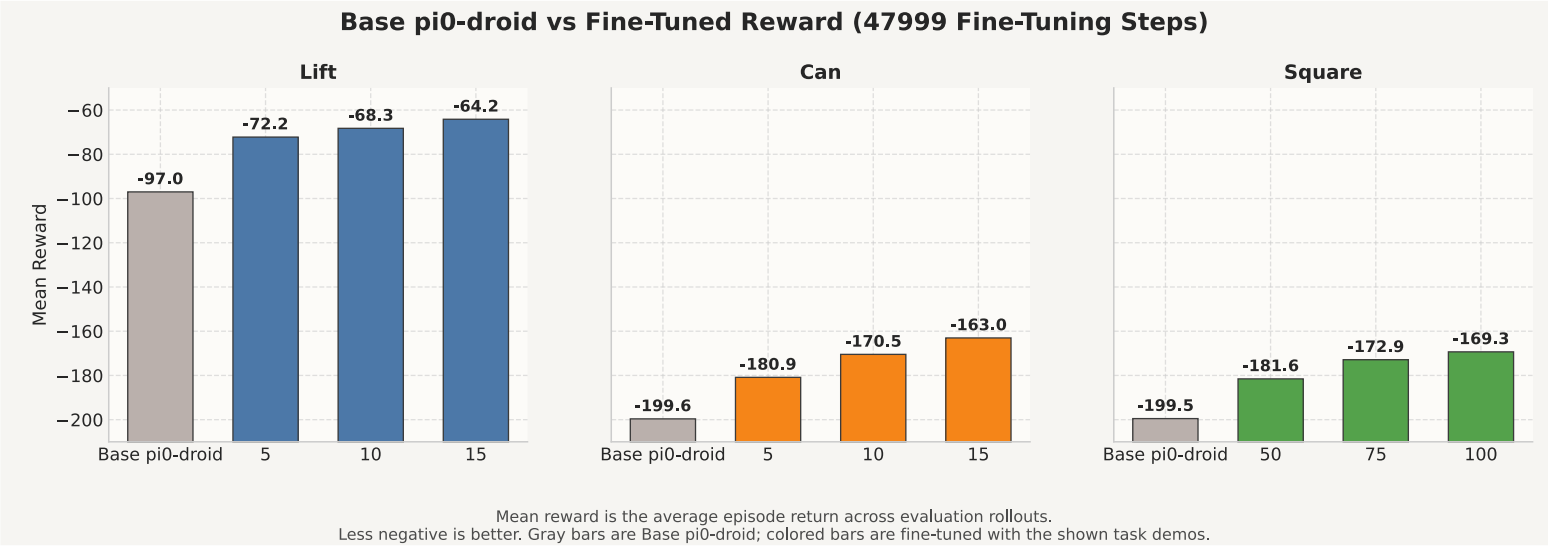


**Figure 5.4: Mean reward after extending fine-tuning to 48,000 steps.**

Figure 5.4 supports the same interpretation. Reward remains clearly better than the zero-shot base policy across tasks, including Square, but the longer run is not uniformly better than the main 24,000-step setting. In particular, the best Square reward at 48,000 steps remains strong but does not overturn the main milestone conclusion.

For that reason, the 48,000-step experiments are best interpreted as a diagnostic extension rather than the main benchmark result. They confirm that longer fine-tuning alone was not enough to close the remaining gap, especially on the precision-sensitive Square task.

### 5.8 Interpreting Square: Progress Without Completion

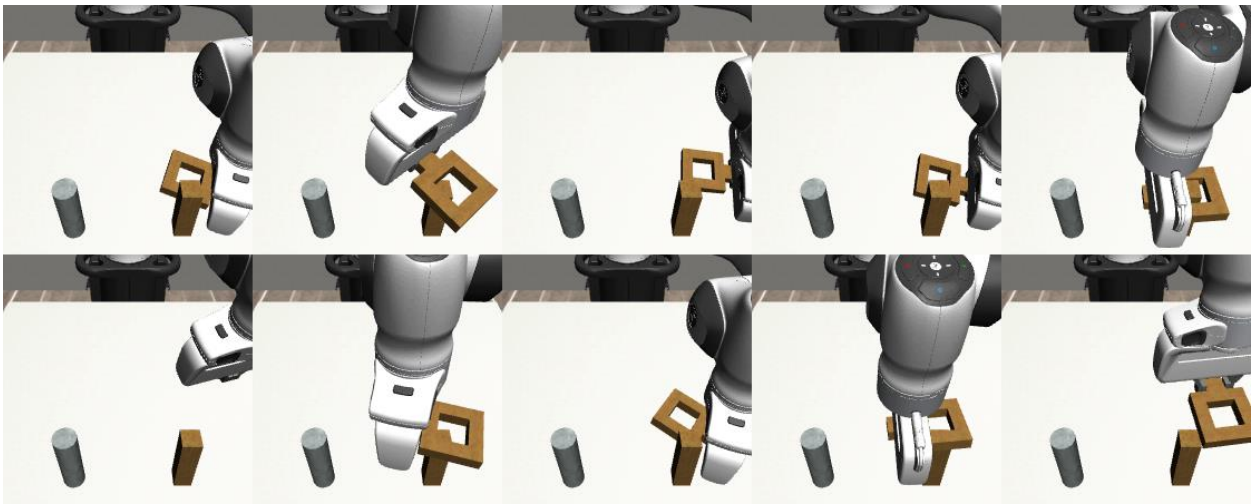

**Figure 5.5: Representative Square rollout after extended fine-tuning, showing near completion without task success.**

Figure 5.5 helps explain why the Square success score does not tell the whole story. Across both fine-tuning budgets, Square remained at zero binary success, but the reward trends showed clear improvement. The qualitative rollout frames support that interpretation: the policy is visibly closer to the correct interaction pattern and near-final insertion state than the zero-shot policy, but it still does not complete the final alignment quickly enough within the allotted time.

This is an important result for the thesis. It suggests that the remaining failure mode is not broad task misunderstanding. Instead, it is a narrower gap between improved nominal behavior and reliable completion under tight timing and precision constraints. That is exactly the type of gap for which SAILOR's recovery-oriented search is most relevant.

### 5.9 Interpretation and Takeaways

Milestone 3 produced four main conclusions.

First, JAX LoRA was the right engineering choice for this stage. It matched the practical fine-tuning path available in OpenPI, avoided turning the project into a full-model training systems exercise, and kept the milestone focused on task alignment rather than framework reconstruction [29], [41].

Second, the custom RoboMimic HDF5 bridge was justified. By keeping the training path anchored to the same benchmark data family and the same validated observation and action contract from Chapter 4, the milestone reduced ambiguity about what had changed between zero-shot inference and offline fine-tuning [40]–[43].

Third, fine-tuning clearly improved $\pi_0$-Droid as a base policy. Lift improved strongly, Can improved more modestly, and Square improved in reward even though it remained unsuccessful under the binary task criterion.

Fourth, fine-tuning alone did not close the remaining completion gap on the hardest task. On Square, the main limitation was not lack of progress, but lack of reliable completion under the fixed episode limits. That result motivated the next milestone. By the end of Chapter 5, $\pi_0$-Droid had become a stronger task-aligned base policy, but the hardest task still exposed a gap between improved nominal behavior and reliable task completion. Chapter 6 therefore tests whether adding SAILOR's recovery mechanism changes that outcome.

### 5.10 Chapter Summary

This chapter presented the task-alignment stage for $\pi_0$-Droid using JAX LoRA fine-tuning on RoboMimic demonstration data. The initial plan of converting to PyTorch and fine-tuning there was replaced by a JAX-native LoRA path because it was more practical, more compute-efficient, and better aligned with the project scope. A custom RoboMimic-to-OpenPI training bridge was then built so that fine-tuning could operate directly on the same HDF5-style benchmark data family used elsewhere in the thesis while preserving the validated observation and action contract from Chapter 4 [40]–[46].

Under the main 24,000-step fine-tuning budget, $\pi_0$-Droid improved substantially on Lift and more modestly on Can, while Square remained unsuccessful in binary terms but improved clearly in mean reward. The 48,000-step extension confirmed that longer training was not always better and did not fundamentally change the outcome on Square. The main outcome of Milestone 3 is therefore that $\pi_0$-Droid could be made meaningfully task-aligned before SAILOR integration, while the hardest task still exposed a gap between improved nominal behavior and reliable completion under fixed episode limits. This established the starting point for Chapter 6: whether SAILOR's recovery and relabeling loop could turn that improved but incomplete behavior into more reliable task completion.

# 6 Integrating $\pi_0$-Droid as the SAILOR Base Policy

## 6.1 Milestone Objective

The fourth milestone of this thesis was to integrate the task-aligned $\pi_0$-Droid policy into the full SAILOR training and evaluation loop as the base policy. Chapters 4 and 5 had already established the two prerequisites for this stage. First, $\pi_0$-Droid could run correctly inside RoboMimic under a validated observation and action interface. Second, JAX LoRA fine-tuning could adapt $\pi_0$-Droid into a substantially stronger task-aligned policy on RoboMimic tasks. The remaining question was therefore whether SAILOR's latent-search recovery mechanism could still add useful corrections when the nominal action proposals came from a stronger pretrained Vision–Language–Action policy rather than from a diffusion policy.

This milestone addressed the thesis question in its most direct form. The goal was not to redesign SAILOR, but to replace the diffusion-policy source of nominal action proposals with $\pi_0$-Droid while preserving the rest of the system as cleanly as possible. In practical terms, this meant keeping SAILOR's search, world model, reward model, critic, warm-start procedure, and round-based training logic as unchanged as possible, and then testing whether search-based recovery still added value once the base policy was no longer a narrow task-specific diffusion model.

This stage was also the most difficult experimentally. Once the observation and action interfaces had been validated in the earlier milestones, the main difficulty was no longer conceptual compatibility. Instead, it became a systems and optimization problem: how to make a much larger VLA-based SAILOR run train, resume, and produce stable results on shared Compute Canada infrastructure under multi-day wall-time constraints.

## 6.2 Integration Strategy

The integration strategy was intentionally conservative. The objective was not to redesign SAILOR around $\pi_0$-Droid, but to mirror the role previously played by diffusion policy as closely as possible. In practical terms, this meant carrying forward the validated action contract from Chapter 4 and the fine-tuned JAX LoRA checkpoint from Chapter 5, while changing only the code paths needed to let $\pi_0$-Droid provide the nominal chunk proposals inside SAILOR.

This distinction matters for interpretation. The contribution of Milestone 4 is not a new planning algorithm. SAILOR remained the same residual model-predictive control layer: at each environment step, the base policy proposes a short action chunk; overlapping chunk predictions

are smoothed inside the base-policy wrapper; SAILOR then samples residual corrections, evaluates corrected trajectories in the learned world model using the reward model and critic, executes only the first corrected action, and replans at the next step [4]. What changed was the origin of the nominal chunk and the path by which corrected trajectories were later distilled back into the base policy.

To keep the comparison fair, most of the reproduced SAILOR settings were deliberately left unchanged. Warm-start percentage, online training structure, rollout logic, per-episode time limits, and the broader planning framework were inherited from the reproduced diffusion-policy baseline [4]. The main intentional differences were the $\pi_0$-specific base-policy backend, the 224×224 visual path required by $\pi_0$-Droid, and the JAX update route used to update $\pi_0$-Droid from relabeled SAILOR trajectories.

### 6.3 Code-Level Integration

The integration was implemented through coordinated changes across the local Sailor_Pi0 repository and the modified SAILOR codebase used as a submodule in this project [47]–[51]. On the local project side, the main additions were pi0_action_contract.py, pi0_runtime_bridge.py, pi0_round_dataset.py, and pi0_jax_update_runtime.py [47]. These files carried the validated $\pi_0$ interface into the full SAILOR loop by handling action semantics, simulator bridging, relabeled round-dataset construction, and JAX-based $\pi_0$ updates.

On the SAILOR side, a dedicated pi0_jax backend was added so that $\pi_0$-Droid could replace diffusion policy as the nominal chunk generator inside the existing recovery loop [49]. The environment wrapper and supporting utilities were extended to expose the same 8-dimensional joint-velocity-plus-gripper control path and $\pi_0$-specific proprioceptive fields already validated in Chapter 4 [34]–[36], [49]. This preserved interface consistency across inference, fine-tuning, and integration.

The largest model-side change was the visual path. The reproduced diffusion-policy baseline used 64×64 images, whereas the validated $\pi_0$ runtime used 224×224 inputs. To keep the integrated system consistent, the Dreamer encoder and decoder in networks.py were extended to support the 224×224 regime [47], [49]. In that sense, the change was mainly a front-end adaptation for the 224 × 224 $\pi_0$-Droid visual path rather than a redesign of the Dreamer-style latent dynamics.

Finally, the original diffusion-policy distillation path was replaced with a $\pi_0$-specific JAX update route through pi0_jax_update_runtime.py, which reused the LoRA fine-tuning infrastructure built in Chapter 5 [41], [47]. Additional launch and resume scripts were then later added so that these long integrated runs could be executed and recovered on shared Compute Canada resources [48], [50], [51].

### 6.4 Training Setup, Compute Cost, and Comparison Views

The integrated $\pi_0$-Droid + SAILOR experiments used the same RoboMimic task family as the earlier milestones: Lift, Can, and Square [3], [4]. For Lift and Can, the integrated runs reported here use the same 5-, 10-, and 15-demonstration settings as the earlier stages. For Square, the currently available integrated runs cover 50 and 75 demonstrations. The 100-demo Square integrated run was not yet available in the same form at the time of analysis, and is therefore shown as unavailable in the corresponding summary plots.

To keep the comparison aligned with earlier chapters, the same episode limits were retained throughout the integrated runs: 100 environment steps for Lift and 200 for Can and Square [4], [39]. These were the same task-level horizons used in the earlier $\pi_0$ stages, so later differences should be interpreted as policy and recovery differences rather than as changes in benchmark difficulty or evaluation budget.

These integrated runs were the most expensive experiments in the thesis. In practice, stable execution required roughly 65 GB of GPU memory, approximately 128 GB of system memory for safe execution, and one CPU core on shared Compute Canada infrastructure. Individual runs typically required between five and seven days of wall-clock time. Under these conditions, Lift was the only task that consistently progressed far enough to complete the intended SAILOR schedule within the available wall-time window. The Can and Square runs often terminated before the full planned budget was reached, but they still produced informative intermediate checkpoints and therefore usable partial training curves. This made Milestone 4 not only a model-integration problem, but also a systems problem involving long-job launch scripts, checkpoint recovery, and resume logic [48], [50], [51].

Securing and maintaining these long runs on shared infrastructure was itself difficult, and in practice the experiments required checkpoint migration, resume logic, and close monitoring of failures and wall-time expirations.

The integrated runs were evaluated at initialization and then every ten SAILOR rounds [48]. Because several runs could not be carried to identical completion under shared-cluster wall-time limits, the results are summarized through four complementary comparison views. First, step-init versus best-round plots show the strongest performance reached within a selected run. Second, step-init versus latest-round plots show whether those gains were still present at the latest available round. Third, matched-round baseline comparisons evaluate DP + SAILOR at the same round index as the best $\pi_0$ + SAILOR round, giving the fairest direct comparison to the original SAILOR formulation. Fourth, best-available baseline comparisons place the strongest diffusion-policy result against the strongest integrated $\pi_0$-based result, which is less controlled but still useful as an upper-bound gap view.

Across all figures, results are reported using both success and mean reward. Success remains the main benchmark metric, but reward is especially useful on harder tasks where binary completion remains low or zero, because it shows whether the integrated system is still making partial progress toward the task.

### 6.5 Best-Round Integrated Results

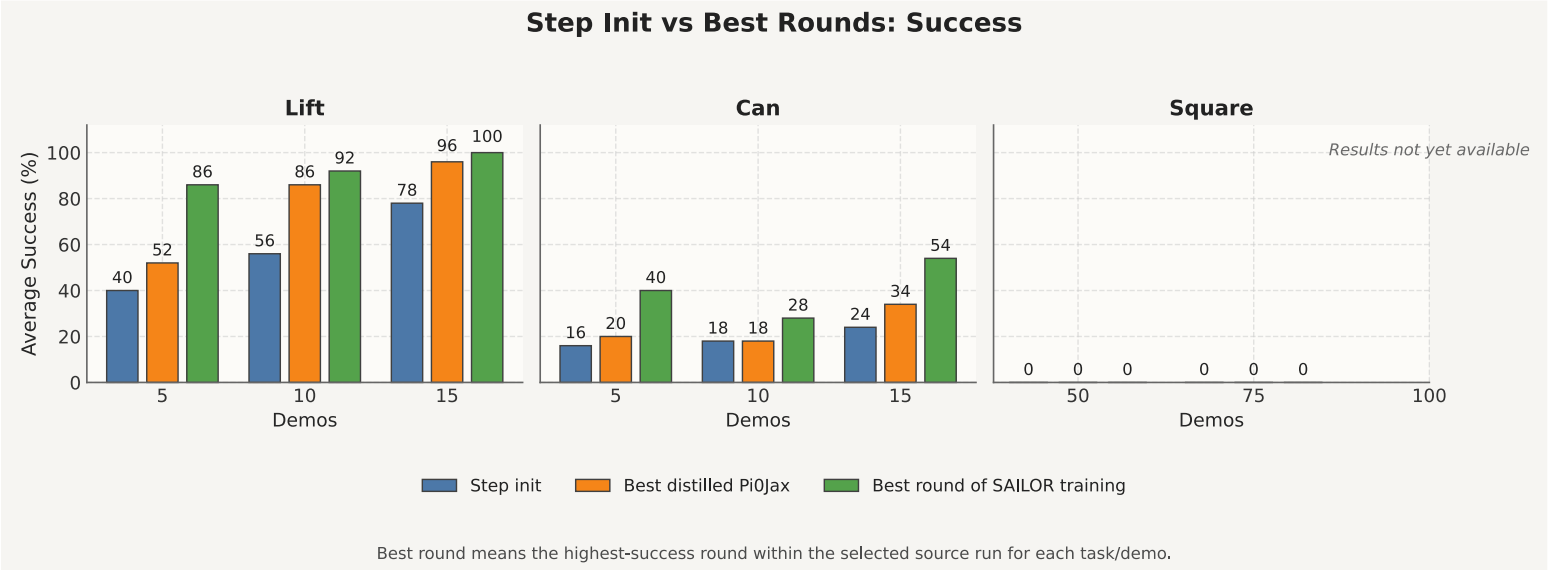


**Figure 6.1: Best-round success for integrated $\pi_0$-Droid + SAILOR runs, compared against the integrated initialization point and the best distilled $\pi_0$-Droid checkpoint within the selected run.**

Figure 6.1 gives the clearest positive result of Milestone 4. Once $\pi_0$-Droid is placed inside SAILOR, the planner can still improve performance beyond the initialized base policy. The pattern is strongest on Lift, where the integrated system improves consistently from initialization to the best distilled checkpoint and then again to the best SAILOR round. That is the expected pattern if the integrated system is functioning properly: the task-aligned $\pi_0$-Droid base is already useful, online updates strengthen it further, and SAILOR's residual search still adds additional recovery on top.

Can shows the same qualitative pattern, but less strongly. The best-round view still shows improvement beyond initialization, and in the stronger settings it also improves beyond the best distilled checkpoint alone. This suggests that the gains are not coming only from additional $\pi_0$-Droid updates, but from the full SAILOR loop operating on top of the updated base policy.

Square remains unresolved in binary success in the currently available runs. That does not make the figure uninformative. It shows instead that the current integrated setup is not yet strong enough to turn partial progress into reliable completion on the hardest task.

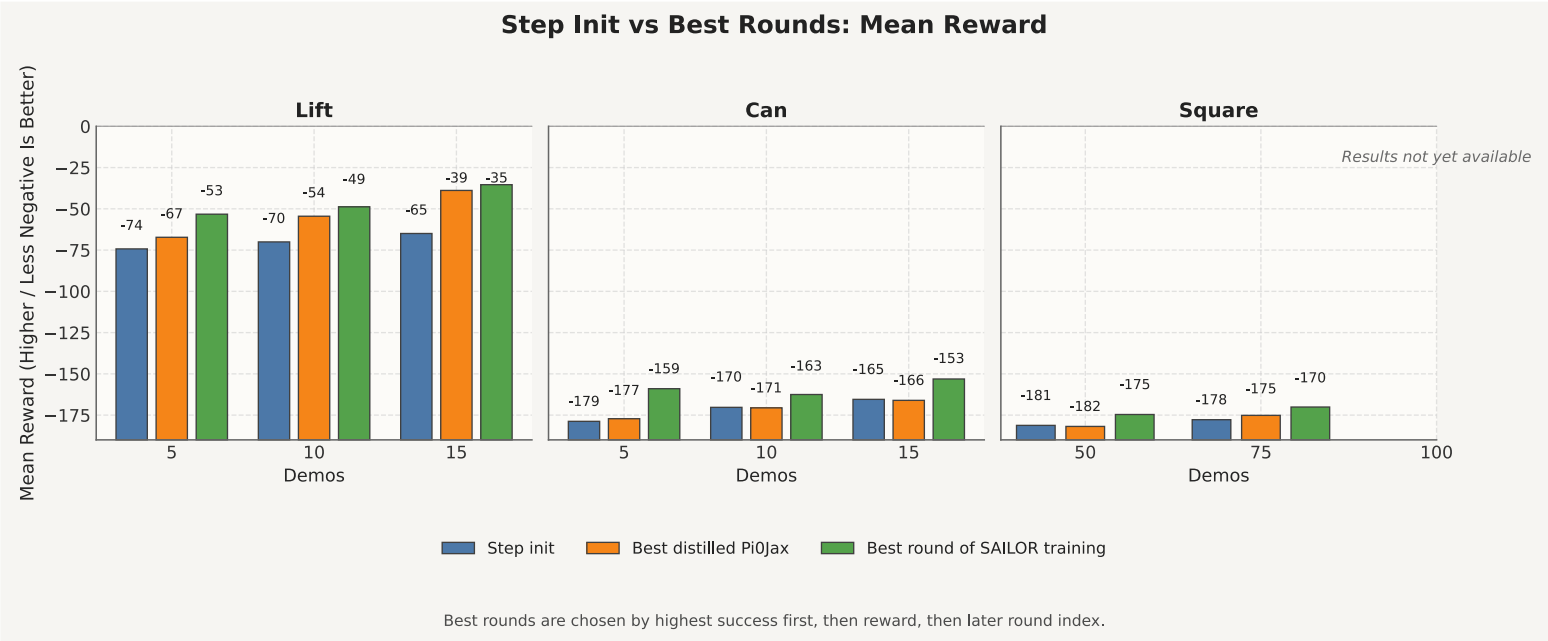


**Figure 6.2: Best-round mean reward for integrated $\pi_0$-Droid + SAILOR runs. Less negative reward indicates better task progress.**

Figure 6.2 supports the same conclusion using reward. On Lift, the reward trend agrees with the success trend, which shows that the stronger best-round results reflect better overall behavior rather than isolated threshold crossings. On Can, reward is especially useful because the task is not yet saturated; the best-round gains show that the integrated system is making meaningfully better progress, not just occasionally succeeding.

Square is where the reward figure matters most. Success remains at zero, but reward still improves modestly in the best-round view. That makes the more accurate interpretation clear: the integrated system is not completely static on Square, but it still cannot reliably convert partial progress into completed insertions within the fixed task horizon.

Taken together, Figures 6.1 and 6.2 support the core positive claim of this chapter: SAILOR remains useful even when the base policy is no longer a narrow task-specific diffusion model, but a stronger fine-tuned VLA-based policy.

### 6.6 Latest-Round Results and Training Stability

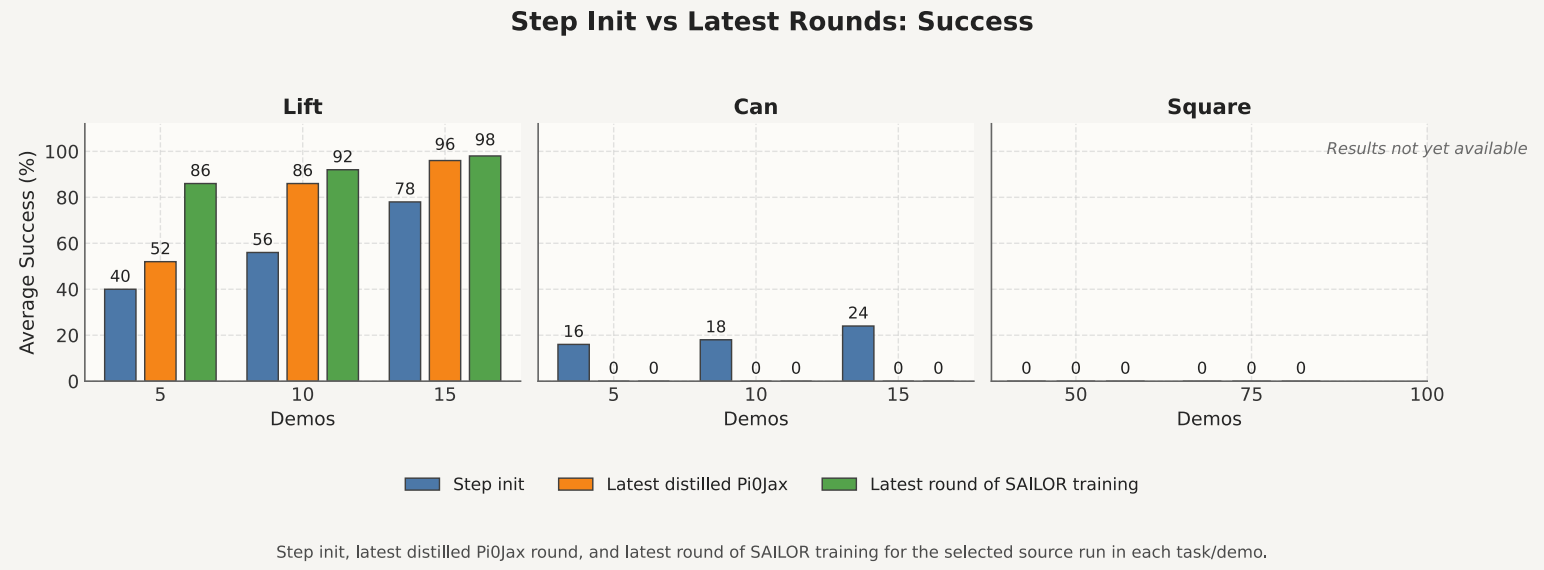


**Figure 6.3: Latest-round success for integrated $\pi_0$-Droid + SAILOR runs, compared against the integrated initialization point and the latest distilled and online checkpoints available in the selected run.**

The best-round view shows what the integrated system could achieve at its strongest point. Figure 6.3 asks the harder question: were those gains still present at the latest available round? Lift remains comparatively healthy under this stricter view. The integrated system keeps most of its

earlier improvement, which suggests that the Lift setting is not only effective but also relatively stable.

Can behaves very differently. In the best-round view, the integrated system showed meaningful gains. In the latest-round view, those gains largely disappear. This means the system could discover stronger behavior on Can, but it was not yet training stably enough to preserve that behavior through later rounds. The issue on Can is therefore not simply lack of capability. It is instability.

Square again remains unresolved, and the latest-round view reinforces that the harder settings were not merely slightly weaker. Under the current schedule, the integrated system still could not sustain reliable completion.

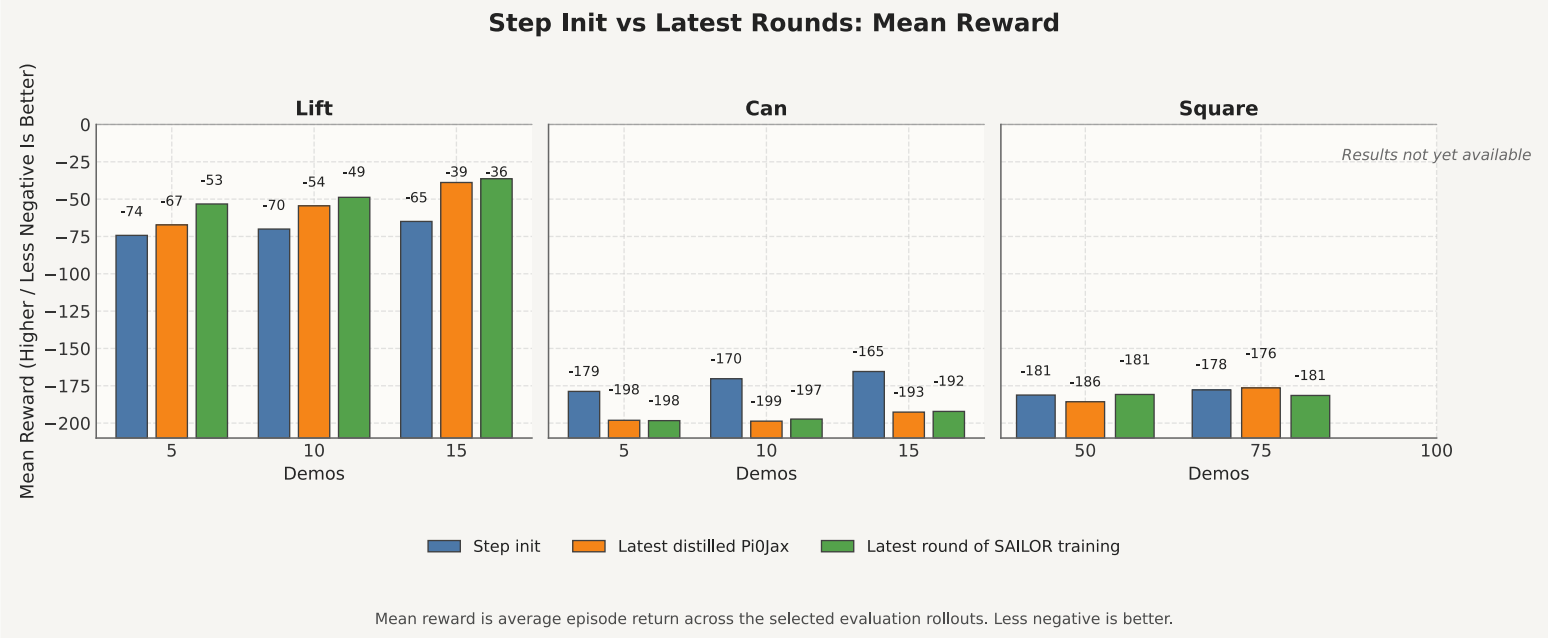


**Figure 6.4: Latest-round mean reward for integrated $\pi_0$-Droid + SAILOR runs. Less negative reward is better.**

Figure 6.4 sharpens the same interpretation. Lift keeps much of its reward improvement even at the latest checkpoint, which supports the conclusion that the integrated Lift runs are both effective and comparatively stable. Can, by contrast, loses a large part of the earlier best-round improvement by the latest checkpoint, which matches the collapse seen in success. Square remains weak and shows little convincing evidence of stable late-round improvement.

Together, Figures 6.3 and 6.4 show why Milestone 4 should be described as promising but not yet finished. The best-round figures demonstrate that the integration can work. The latest-round figures show that longer training was not always better, especially on the harder tasks. At this point, the main limitation is no longer interface compatibility. It is optimization and stability in the integrated training loop.

### 6.7 Matched-Round Comparison Against the Diffusion-Policy Baseline

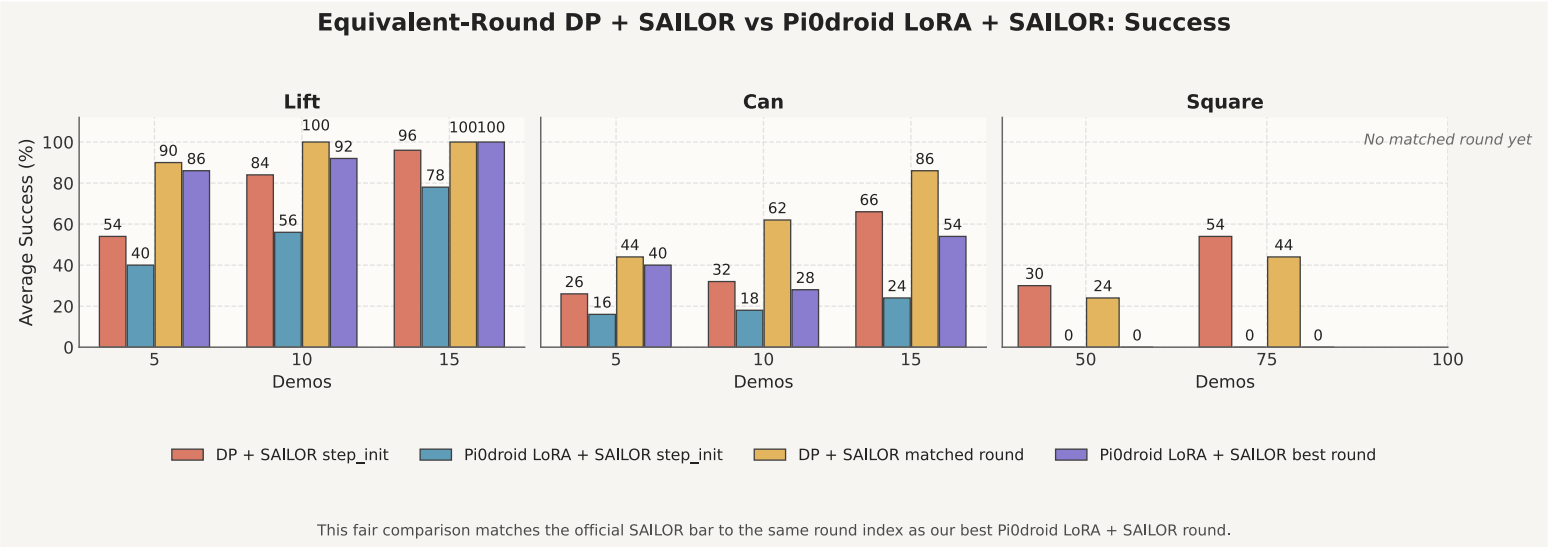


**Figure 6.5: Matched-round success comparison between DP + SAILOR and $\pi_0$-Droid LoRA + SAILOR. The diffusion-policy baseline is evaluated at the same round index as the best $\pi_0$-Droid + SAILOR round.**

Figure 6.5 is the fairest direct baseline comparison in the chapter. Rather than comparing each system at its own best observed point, it aligns the reproduced DP + SAILOR result to the same round index at which the integrated $\pi_0$-Droid + SAILOR system reached its best result. This removes the clearest source of unfairness from the comparison.

Under this matched-round view, the diffusion-policy baseline still remains stronger overall. The gap is smallest on Lift, where the integrated $\pi_0$-Droid system is already in a competitive regime and reaches parity at the highest-data setting. The gap is more substantial on Can, where DP + SAILOR remains clearly ahead even when compared at the same training stage. Square remains the clearest hard case, with the diffusion-policy baseline already showing nonzero success while the integrated $\pi_0$-Droid system remains unsuccessful in the currently available runs.

This is the fairest direct baseline comparison in the chapter. It supports a careful claim: the integrated $\pi_0$-Droid + SAILOR system is clearly viable and already competitive on Lift, but under the current benchmark setting it does not yet surpass the reproduced diffusion-policy baseline overall. Given the conservative tuning used here, the fixed episode limits, and the fact that $\pi_0$-Droid originates from real-world data rather than this simulator, this result should be interpreted as an optimization and transfer-gap result rather than as evidence that a VLA base policy is fundamentally weaker.

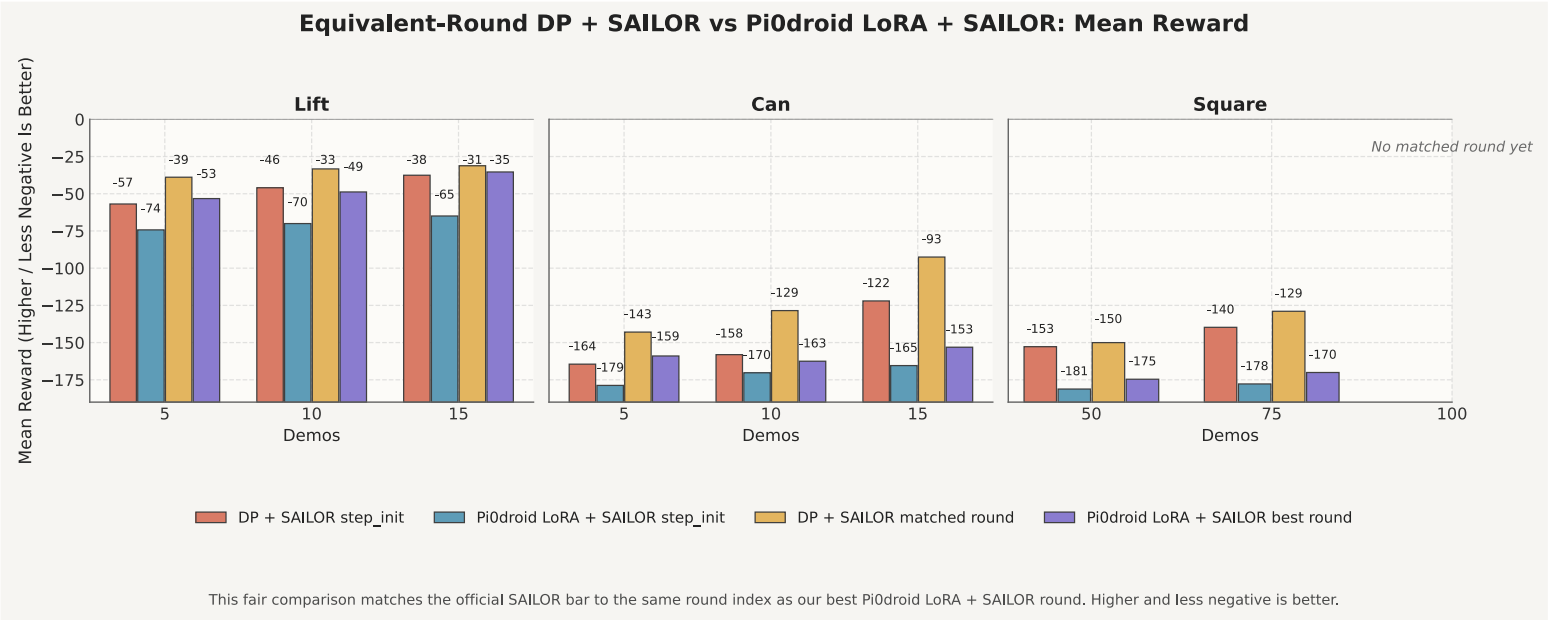


**Figure 6.6: Matched-round mean reward comparison between DP + SAILOR and $\pi_0$-Droid LoRA + SAILOR. Less negative reward is better.**

Figure 6.6 tells the same story through reward. Lift lies in a similar qualitative performance regime even when the diffusion-policy baseline remains somewhat stronger. Can still shows a meaningful gap, and the reward view makes clear that this gap is not only about final binary success. The reproduced diffusion-policy baseline is also making stronger task progress within the same round budget. Square remains the clearest hard case: the $\pi_0$-Droid system shows limited progress, but the diffusion-policy baseline remains clearly more effective even under matched-round comparison.

These matched-round results are important because they separate two claims that might otherwise be blurred together. The integrated $\pi_0$-Droid + SAILOR system is real and experimentally meaningful. It is not yet stronger than the reproduced diffusion-policy baseline.

### 6.8 Best-Available Comparison Against DP + SAILOR

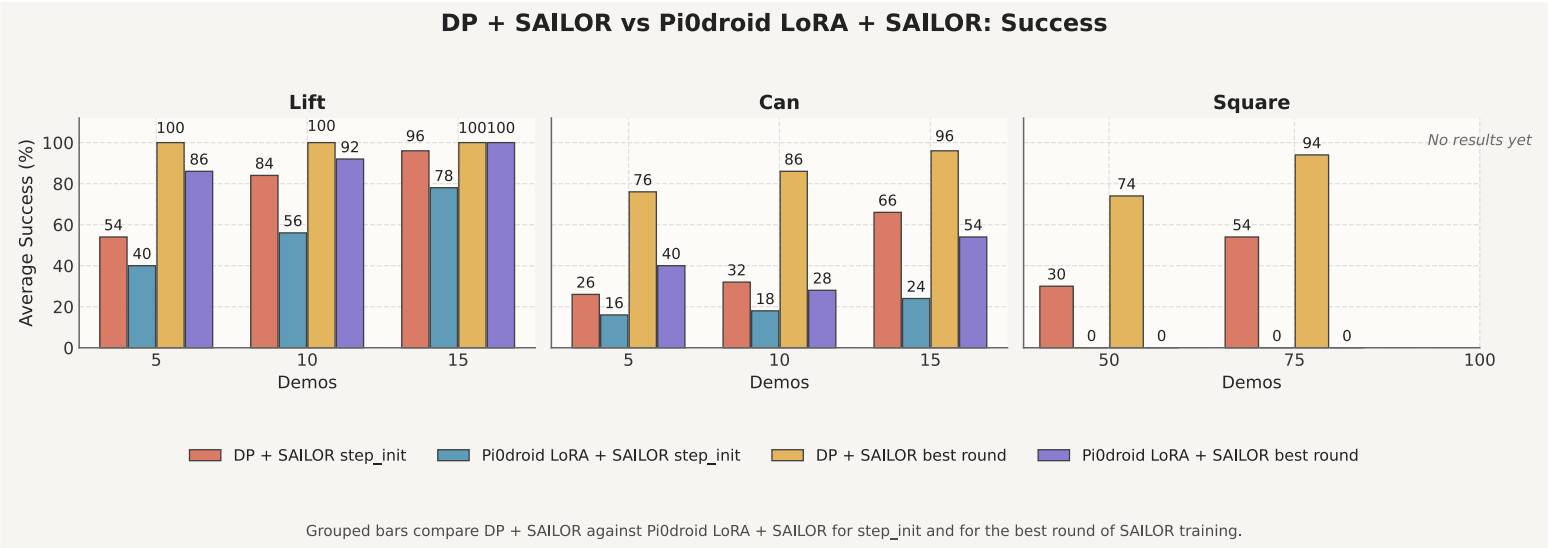


**Figure 6.7: Best-available success comparison between the reproduced DP + SAILOR baseline and the integrated $\pi_0$-Droid LoRA + SAILOR system.**

For completeness, Figure 6.7 compares the strongest available integrated $\pi_0$-Droid + SAILOR result against the strongest available DP + SAILOR result without matching round indices. This is a looser comparison than the matched-round view, but it is still useful because it shows the current benchmark gap to the more mature diffusion-policy system.

The broad picture is consistent with the matched-round analysis, but more extreme. Lift is the closest task, and at the highest-data setting the integrated $\pi_0$-Droid system effectively reaches the same success ceiling. Can remains clearly behind, and Square remains the task with the largest gap. This figure is less useful for fairness than the matched-round view, but it is still useful for showing the current benchmark gap to the strongest DP + SAILOR result under the present integrated setup. That gap should be read cautiously, since the $\pi_0$-Droid system was evaluated here under fixed simulator horizons and without a full VLA-specific tuning study.

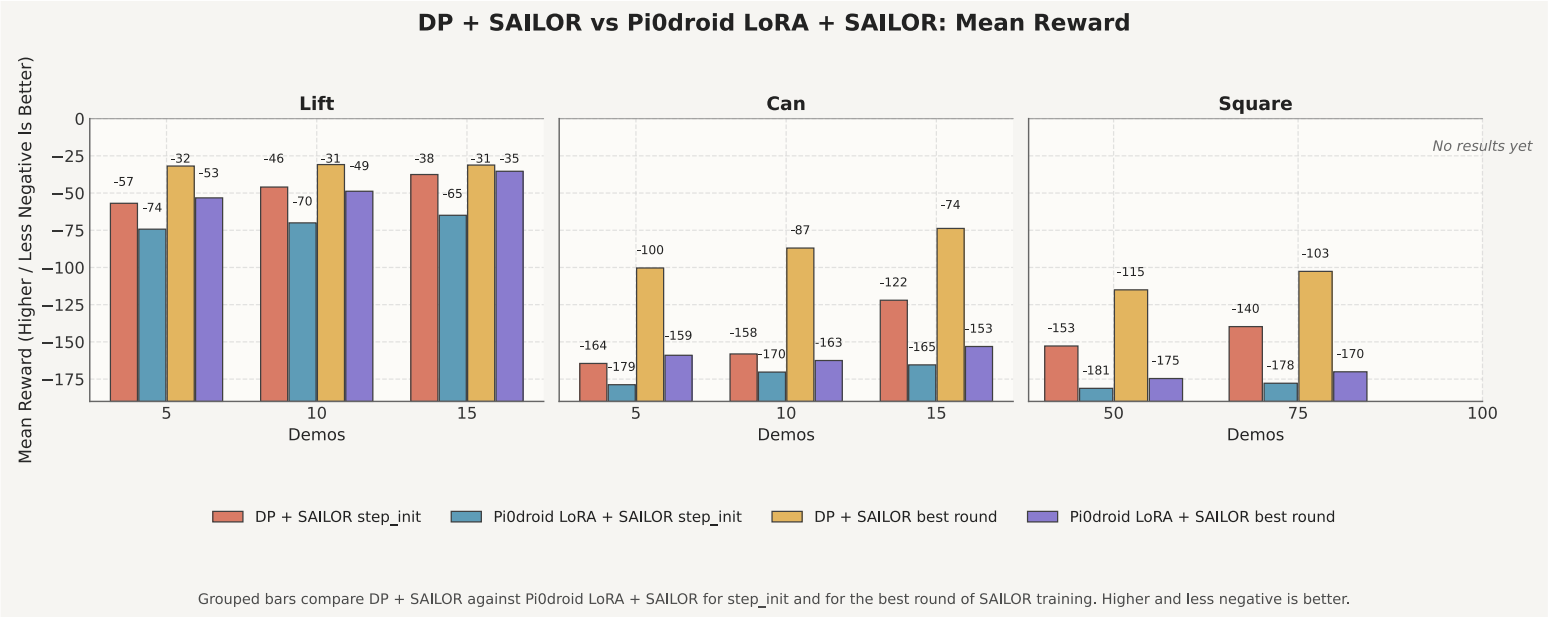


**Figure 6.8: Best-available mean reward comparison between the reproduced DP + SAILOR baseline and the integrated $\pi_0$-Droid LoRA + SAILOR system. Less negative reward is better.**

Figure 6.8 supports the same interpretation through reward. The integrated $\pi_0$-Droid + SAILOR system is closest to DP + SAILOR on Lift, farther away on Can, and still clearly limited on Square. This reward view is useful because it shows that the remaining gap is not only about final success rates. The reproduced diffusion-policy system also tends to make stronger progress throughout the episode, which suggests that the present $\pi_0$-Droid-based system is still less efficient and less stable under the same fixed control budget.

### 6.9 Interpretation and Takeaways

Milestone 4 leads to four main conclusions.

First, the integration worked. $\pi_0$-Droid could be inserted into the full SAILOR loop as the base policy while preserving SAILOR's latent-space search, reward-guided evaluation, critic guidance, and per-step replanning structure. More importantly, the integrated system was not merely executable. It produced rounds in which SAILOR improved performance beyond both the integrated initialization point and, in the stronger settings, beyond the best distilled $\pi_0$-Droid checkpoint alone.

Second, stronger pretrained priors and search-based recovery appear complementary rather than redundant. This is clearest on Lift and visible more modestly on Can. The best-round plots show that SAILOR can still add useful recovery even when the base policy is already strong and task-aligned. That is the main positive result of the chapter.

Third, the current bottleneck is optimization rather than compatibility. The latest-round figures show that longer training was not always better and that some gains, especially on Can, could be lost later in training. This suggests that the remaining challenge is not whether $\pi_0$-Droid can be integrated into SAILOR, but how the combined system should be tuned and stabilized. Given the high memory demands and multi-day runtimes of these jobs, this optimization problem is inseparable from the practical realities of checkpointing, resume behavior, and long-run experiment management on shared infrastructure.

Fourth, the unresolved tasks point to limits of efficiency and stability rather than to one simple failure mode. Lift is already strong and comparatively stable. Can is promising but fragile, which suggests that the system can reach stronger behavior than it can currently preserve. Square remains the clearest hard case. Across the best-round and reward views, Square shows evidence of partial progress without reliable completion, which suggests that the integrated $\pi_0$-Droid-based system is often moving in the right direction but is still not efficient enough within the fixed episode budget.

Taken together, these results support a qualified but meaningful claim. The integrated $\pi_0$-Droid + SAILOR system is already a real and promising alternative to the original diffusion-policy baseline, but under the present constraints it is not yet a fully tuned replacement.

### 6.10 Chapter Summary

This chapter presented the final milestone of the thesis: integrating $\pi_0$-Droid into the full SAILOR loop as the base policy. The integration reused the validated action contract and 224 × 224 visual path from the earlier milestones, added a dedicated $\pi_0$ backend and update route, extended the Dreamer image pipeline to the $\pi_0$ input size, and replaced diffusion-policy distillation with a JAX LoRA update path built on relabeled SAILOR trajectories [47]–[51].

The main result is that the integration is possible and meaningful. On Lift, and to a lesser extent on Can, SAILOR still improves performance on top of the fine-tuned $\pi_0$-Droid base policy, which shows that stronger pretrained priors and latent-search recovery can work together. At the same

time, the results also show that the system is not yet fully tuned. Best-round performance is encouraging, but latest-round performance shows that longer training was not always better, particularly on the harder tasks. Square remains the clearest unresolved case, with evidence of partial progress but not reliable completion.

The most important conclusion of Milestone 4 is therefore not that $\pi_0$-Droid + SAILOR has already reached its full benchmark potential, but that the system can be integrated successfully and can produce useful recovery behavior under a demanding shared-compute setting. At this stage, the main remaining challenge is not basic compatibility, but optimization, stability, and systems tuning for a larger and more expensive integrated VLA-based SAILOR configuration.

# 7 Discussion

## 7.1 Answering the Research Question

This thesis asked whether a pretrained Vision–Language–Action model, specifically $\pi_0$-Droid, could serve as an effective base policy within SAILOR, combining broad pretrained visuomotor priors with latent-search-based recovery and online improvement [2], [4]. The results support a qualified but meaningful answer.

The strongest conclusion is that the combination is real and meaningful. $\pi_0$-Droid was not only made executable in the RoboMimic setting, but could also be task-aligned and then used inside the full SAILOR loop. Once integrated, SAILOR still improved behavior beyond the initialized $\pi_0$-Droid base policy, especially on Lift and, more modestly, on Can [47]–[51]. This shows that a stronger pretrained base policy does not make SAILOR unnecessary. Search-based recovery still adds value.

At the same time, the current integrated system did not yet surpass the reproduced diffusion-policy baseline overall under the benchmark conditions used here. That result should be interpreted carefully. The present system was evaluated under fixed episode limits, conservative fairness-oriented settings, and a relatively restricted update path. In addition, $\pi_0$-Droid is rooted in real-world robot data [1], [2], whereas the final benchmark in this thesis is RoboMimic simulation [3]. A real-to-sim transfer gap is therefore a plausible part of the remaining performance gap.

The most accurate answer is therefore this: $\pi_0$-Droid can serve as a meaningful SAILOR base policy, and SAILOR can still help on top of it. The main remaining limitation is not basic

compatibility, but how to tune, adapt, and stabilize the combined system so that its strengths appear consistently under the simulator's fixed horizons.

### 7.2 What the Results Actually Changed

The most important outcome of the thesis is not a single benchmark number. It is that the nature of the problem changed over the course of the work.

At the beginning, the main uncertainty was structural. It was not clear whether a DROID-aligned VLA could be inserted fairly into the RoboMimic and SAILOR setting at all. There were obvious reasons to doubt it: different control conventions, different image preprocessing assumptions, different gripper semantics, different update paths, and a base model trained on real-world data rather than on the benchmark itself.

By the end of the thesis, that uncertainty had been reduced substantially. The interfaces were aligned, the policy could be fine-tuned on the benchmark task family, and the integrated system produced clear cases in which SAILOR improved performance on top of the VLA base. That means the central question is no longer whether the two ideas fit together in principle. They do.

What remains unresolved is a narrower and more interesting question: how should a larger VLA-based SAILOR system be tuned so that the benefits seen in the best rounds become stable and repeatable? That is a better problem to have. It means the thesis moved the project from feasibility to optimization.

### 7.3 Why the Current Gap Still Exists

The remaining gap to the reproduced diffusion-policy baseline appears to come from several interacting causes rather than one single failure.

The first is that the current integrated system is almost certainly under-tuned for its regime. Many SAILOR settings were intentionally kept close to the reproduced diffusion-policy baseline in order to preserve fairness and isolate the integration question [4]. That was the right methodological choice for a first integration study. However, a larger VLA base policy is not just a stronger version of the same object. It is a different regime, with different adaptation behavior, different failure modes, and likely different scheduling needs.

The second is that the present $\pi_0$-Droid update path remained relatively limited. LoRA was a practical choice and the correct one for this thesis under one-GPU constraints, but it also restricted how the model could adapt. The pattern seen in Chapter 6, where the best observed round was often better than the latest one, especially on Can, is consistent with a system that can reach stronger behavior but cannot yet preserve it robustly.

The third is likely transfer. $\pi_0$-Droid inherits priors from real-world DROID data [1], while the final benchmark here is simulated [3]. Even with the same Franka embodiment, differences in image statistics, contact behavior, timing, and scene structure can matter. This is especially relevant when interpreting Square. A real-world-trained policy that looks qualitatively close but misses the binary threshold in simulation should not be read as broadly confused. It may simply be expressing useful priors inefficiently under a different domain and a strict time budget.

The fourth is efficiency. The reward results suggest that Can and Square were often closer than their binary success scores alone indicate. This means the integrated system was frequently making meaningful progress, but not finishing quickly enough within the fixed horizon. That matters because it changes the interpretation of failure. The current problem often looks less like "the policy does not know what to do" and more like "the policy is not yet reactive, sharp, or stable enough to finish in time."

### 7.4 What Should Be Improved Next

The next improvements are most likely to come from co-tuning $\pi_0$-Droid and SAILOR together rather than trying to "fix" only one side.

The first priority is to retune SAILOR for a larger VLA base policy. The current settings were useful for a fair first comparison, but the results suggest that the online update schedule is probably too aggressive for the $\pi_0$-Droid path. A more conservative schedule, with fewer update steps, more expert-heavy mixing, lower collection noise, and possibly delayed updates, is the clearest next direction. Can is the strongest evidence for this, because it shows that the system can find stronger behavior before losing it.

The second priority is stabilization. EMA, held-out validation, and better checkpoint selection are all justified directly by the current results. When best-round behavior is stronger than the latest checkpoint, model selection is already part of the problem.

The third priority is broader adaptation of $\pi_0$-Droid itself. LoRA was enough to make the integration meaningful, but it may not be enough to get the most out of a large VLA inside an online relabeling loop. Partial unfreezing or fuller fine-tuning may eventually help, especially on the harder tasks, although that should be approached carefully because greater adaptation capacity can also increase instability.

The fourth priority is reactivity. In the current implementation, overlapping chunk predictions are smoothed before SAILOR applies residual search. That likely helps stability, but on contact-sensitive tasks it may also reduce responsiveness. This is one of the places where a VLA-based SAILOR system may need to differ from the diffusion-policy regime instead of inheriting its assumptions.

The fifth priority is task-specific alignment. Since $\pi_0$-Droid is language-conditioned, prompts matter. Since the benchmark is simulator-based, action targets and timing details matter. Since Square repeatedly appears near completion, it is exactly the kind of task where small changes in conditioning, adaptation, or control sharpness may have a large effect.

### 7.5 Broader Meaning

A broader lesson of the thesis is that robustness in robot learning is unlikely to come from one idea alone.

Large pretrained policies are powerful because they bring broad prior experience. But they still remain vulnerable when execution drifts, timing changes, or contact unfolds differently from what the demonstrations implied. Search-based recovery is powerful because it gives the system a way to imagine, evaluate, and correct. But recovery also depends on the quality of the base policy it is correcting.

This thesis suggests that the stronger path is to make those two strengths support one another. A better prior can make recovery easier. A better recovery mechanism can make broader priors more reliable. In that sense, the value of this work is not just in the specific benchmark results. It is in showing that these two directions can be brought into the same control loop in a principled way.

A broader implication is that SAILOR should not be viewed only as a planner layered on top of $\pi_0$-Droid, but also as a mechanism for improving a pretrained VLA itself through correction, relabeling, and distillation. In that sense, the longer-term opportunity is not only better test-time

recovery, but a better way of adapting large robot policies from limited task-specific demonstrations.

### 7.6 Chapter Summary

This chapter interpreted the thesis results rather than repeating them. The main conclusion is that $\pi_0$-Droid and SAILOR are compatible in a meaningful sense: the VLA can serve as the base policy, and SAILOR can still improve performance on top of it. The remaining benchmark gap to the reproduced diffusion-policy baseline is best understood not as a failure of the idea, but as a consequence of under-tuning, transfer difficulty, limited update capacity, and efficiency under fixed horizons. The problem has therefore moved from "can this work?" to "how well can it work once tuned on its own terms?"

## 8 Conclusion

This thesis investigated whether a pretrained Vision–Language–Action model, $\pi_0$-Droid, could serve as the base policy within SAILOR, combining broad pretrained visuomotor priors with latent-search-based recovery for robotic manipulation.

The answer is yes, but not yet in its final form.

The thesis showed that $\pi_0$-Droid can be integrated into the RoboMimic and SAILOR setting in a technically consistent way. It showed that the model can be task-aligned to the RoboMimic benchmark. It also showed that, once integrated, SAILOR still improves behavior on top of the VLA base policy, especially on Lift and to a lesser extent on Can. These findings establish the core feasibility and value of the combination.

At the same time, the thesis also showed that this first integrated system is not yet fully optimized. Under the benchmark conditions used here, the reproduced diffusion-policy baseline remained stronger overall, especially on Can and Square. However, the pattern of results suggests that the main gap is not one of broad incompatibility. It is better understood as a gap in tuning, stability, transfer, and time efficiency. This is particularly important for interpreting the harder tasks, where reward often improved even when binary success did not. Those results suggest a system that is frequently close, but not yet efficient enough to finish reliably within the fixed horizon.

The main contribution of the thesis is therefore not just a new pipeline, but a change in what is now known. It is now clear that a pretrained VLA can function as a SAILOR base policy. It is now

clear that SAILOR's recovery mechanism still matters on top of a stronger prior. It is also now clearer that the value of combining $\pi_0$-Droid with SAILOR is not only improved execution-time recovery, but also the possibility that SAILOR's correction, relabeling, and distillation loop can help produce stronger adapted VLA policies over time.

More broadly, this thesis points toward a direction for robot learning. Collecting more data is powerful, but data alone does not remove the need for correction when execution drifts. Planning and recovery are powerful, but they work best when the nominal proposal is already strong. A robust robot policy may therefore need both: broad experience before deployment, and corrective reasoning during deployment.

In that sense, the lesson of this thesis is not that larger priors replace search, nor that search makes larger priors unnecessary. It is that the two become more useful when they are joined. A robot does not become reliable only by seeing more, and it does not become reliable only by searching harder. It becomes more capable when it can start from broader knowledge and still recover intelligently when that knowledge is not enough.

That is the path this work opens. Not a finished system, but a clearer direction: learn broadly, adapt carefully, and recover wisely.

## 9 References

[1] A. Khazatsky, D. Xu, P. Florence, R. Calandra, and S. Levine, "DROID: A Large-Scale In-the-Wild Robot Manipulation Dataset," arXiv preprint arXiv:2403.12945, 2024.

[2] Physical Intelligence et al., "$\pi_0$: A Vision-Language-Action Flow Model for General Robot Control," arXiv preprint arXiv:2410.24164, 2024.

[3] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Zhang, Y. Jain, B. Zhao, J. Levine, and S. Levine, "RoboMimic: What Matters in Learning from Offline Human Demonstrations for Robot Manipulation," in Proceedings of the Conference on Robot Learning (CoRL), 2021.

[4] A. K. Jain, V. Mohta, S. Kim, A. Bhardwaj, J. Ren, Y. Feng, S. Choudhury, and G. Swamy, "A Smooth Sea Never Made a Skilled SAILOR: Robust Imitation via Learning to Search," arXiv preprint arXiv:2506.05294, 2025.

[5] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011.

[6] C. Chi, A. Xu, R. Zhang, and D. Jayaraman, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," arXiv preprint arXiv:2303.04137, 2023.

[7] S. A. Mehta, A. Zhang, J. Xu, and M. J. Kochenderfer, "Stable-BC: Controlling Covariate Shift with Stable Behavior Cloning," arXiv preprint arXiv:2408.06246, 2024.

[8] P. Karkus, M. Wulfmeier, M. Božič, D. Barrett, and I. Posner, "Beyond Behavior Cloning in Autonomous Driving: A Survey of Closed-Loop Training Techniques," IEEE Transactions on Intelligent Vehicles, vol. 10, no. 2, pp. 215–233, 2025.

[9] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and Acting in Partially Observable Stochastic Domains," Artificial Intelligence, vol. 101, pp. 99–134, 1998.

[10] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba, "Mastering Diverse Domains through World Models," arXiv preprint arXiv:2301.04104, 2023.

[11] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning Latent Dynamics for Planning from Pixels," arXiv preprint arXiv:1811.04551, 2019.

[12] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to Control: Learning Behaviors by Latent Imagination,” in Proceedings of the International Conference on Learning Representations (ICLR), 2020.

[13] N. Hansen, Q. Wu, P. Abbeel, R. Akrour, and M. Jhunjhunwala, “TD-MPC: Temporal Difference Model Predictive Control,” arXiv preprint arXiv:2209.14280, 2022.

[14] D. Hafner, W. Yan, and T. Lillicrap, “Training Agents Inside of Scalable World Models,” arXiv preprint arXiv:2509.24527, 2025.

[15] S. Ross and J. A. Bagnell, “Reinforcement and Imitation Learning via Interactive No-Regret Learning,” arXiv preprint arXiv:1411.5654, 2014.

[16] K. Ahn, A. Bhardwaj, and P. Abbeel, “Model Predictive Control via On-Policy Imitation Learning,” in Proceedings of Machine Learning Research (PMLR), 2023.

[17] P. Swamy, H. Liu, and D. Bagnell, “Learning from Demonstrations by Moment Matching,” in Proceedings of the Conference on Robot Learning (CoRL), 2021.

[18] G. Williams, A. Aldrich, and E. A. Theodorou, “Model Predictive Path Integral Control: From Theory to Parallel Computation,” Journal of Guidance, Control, and Dynamics, vol. 40, no. 2, pp. 344–357, 2017.

[19] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, “Improved Training of Wasserstein GANs,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[20] P. Ball, A. Cully, and M. A. Osborne, “Efficient Exploration with Conservative Critic Ensembles,” arXiv preprint arXiv:2302.01751, 2023.

[21] L. Chen, A. Kumar, and P. Abbeel, “Randomized Ensembles for Robust Reinforcement Learning,” arXiv preprint arXiv:2110.12563, 2021.

[22] A. Brohan et al., “RT-2: Vision–Language–Action Models from Web-Scale Data,” arXiv preprint arXiv:2307.15818, 2023.

[23] I. Radosavovic, P. Dollár, and R. Girshick, “OpenVLA: An Open Vision–Language–Action Model for Universal Robots,” arXiv preprint arXiv:2401.07969, 2024.

[24] H. Zeng, J. Wu, C. Fang, and J. Malik, “Helix: A Generalist Vision–Language–Action Model for Humanoid Control,” arXiv preprint arXiv:2502.06112, 2025.

[25] Y. Lu, J. Fu, G. Tucker, X. Pan, E. Bronstein, R. Roelofs, B. Sapp, B. White, A. Faust, S. Whiteson, D. Anguelov, and S. Levine, “Imitation Is Not Enough: Robustifying Imitation with Reinforcement Learning for Challenging Driving Scenarios,” IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2023.

[26] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, et al., “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm,” Science, vol. 362, no. 6419, pp. 1140–1144, 2018.

[27] A. Brohan, N. Brown, J. Carbajal, et al., “RT-1: Robotics Transformer for Real-World Control at Scale,” Robotics: Science and Systems (RSS), 2023. (arXiv:2212.06817)

[28] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv preprint arXiv:2506.01844, 2025.

[29] Physical Intelligence, “OpenPI Repository,” GitHub repository. Available: https://github.com/Physical-Intelligence/openpi

[30] Physical Intelligence, “OpenPI DROID Inference Example,” GitHub repository. Available: https://github.com/Physical-Intelligence/openpi/blob/main/examples/droid/main.py

[31] DROID Dataset, “Hardware Setup,” DROID Dataset Documentation. Available: https://droid-dataset.github.io/droid/docs/hardware-setup

[32] DROID Dataset, “DROID Dataset Project Page.” Available: https://droid-dataset.github.io/

[33] P. Ishak, “$\pi_0$-Droid Trial Inference Script for Compute Canada Runtime Validation,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/ee8c46ad7caa6f9349645de9189c8ca95caec8d4/pi0_trial.py

[34] P. Ishak, “Joint-Velocity $\pi_0$-Droid Evaluation Runtime in RoboMimic,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/ee8c46ad7caa6f9349645de9189c8ca95caec8d4/pi0_droid_jointvel_eval_final.py

[35] P. Ishak, “Joint-Velocity $\pi_0$-Droid Runtime with Gripper Reset and Interface Fixes,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/main/pi0_joint_vel_final_simple_fix.py

[36] P. Ishak, “Joint-Velocity $\pi_0$-Droid Evaluation Runtime with 224×224 Input and PyTorch/LoRA-Compatible Loading,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/main/pi0_joint_vel_final_simple_fix_pytorch_lora_eval224_full.py

[37] ARISE Initiative, “Controllers,” robosuite documentation. Available: https://robosuite.ai/docs/modules/controllers.html

[38] ARISE Initiative, “robosuite.models.grippers package,” robosuite documentation. Available: https://robosuite.ai/docs/source/robosuite.models.grippers.html

[39] ARISE Initiative, “robosuite.environments.manipulation package,” robosuite documentation. Available: https://robosuite.ai/docs/source/robosuite.environments.manipulation.html

[40] P. Ishak, “RoboMimic Loader Sanity Check for $\pi_0$-Droid Fine-Tuning,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/sanity_pi0_robomimic_loader.py

[41] P. Ishak, “JAX LoRA Fine-Tuning Script for $\pi_0$-Droid on RoboMimic,” GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/train_pi0_droid_lora_robomimic.py

[42] P. Ishak, “OpenPI RoboMimic Data Configuration for HDF5 Fine-Tuning,” GitHub repository. Available:

https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/openpi_robomimic_data_config.py

[43] P. Ishak, "OpenPI RoboMimic HDF5 Dataset Adapter," GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/openpi_robomimic_hdf5_dataset.py

[44] P. Ishak, "48,000-Step Resume Script for $\pi_0$-Droid LoRA Fine-Tuning," GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/sh_2026_03_02_end_mar/train_pi0_droid_lora_48000_resume_base.sh

[45] P. Ishak, "Sweep Submission Script for Extended $\pi_0$-Droid LoRA Fine-Tuning," GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/sh_2026_03_02_end_mar/submit_pi0_droid_lora_48000_resume_sweep.sh

[46] P. Ishak, "Initial Lift-5 $\pi_0$-Droid LoRA Fine-Tuning Batch Script," GitHub repository. Available: https://github.com/piroo8/Sailor_Pi0/blob/799b06b78d4be9f898428c8f344582b559c13e3d/sh_2026_03_02_end_mar/train_pi0_droid_lora_lift5.sh

[47] P. Ishak, "$\pi_0$ integration updates in Sailor_Pi0," GitHub commit. Available: https://github.com/piroo8/Sailor_Pi0/commit/ee8c46ad7caa6f9349645de9189c8ca95caec8d4

[48] P. Ishak, "Initial SLURM launch scripts for $\pi_0$ + SAILOR runs," GitHub commit. Available: https://github.com/piroo8/Sailor_Pi0/commit/2f7c15cc6cd6006b09f88dee39e9b31f9262b8fd

[49] P. Ishak, "SAILOR integration updates for the $\pi_0$ backend," GitHub commit. Available: https://github.com/piroo8/SAILOR/commit/1c730c550877c1a0ee38ef30733193493bed0a19

[50] P. Ishak, "Resume-run checkpoint recovery for extended $\pi_0$ training," GitHub commit. Available: https://github.com/arnavkj1995/SAILOR/commit/ab0e78afea843f6a646394ea46c5ae3ca2d0e051

[51] P. Ishak, “Checkpoint-24000 resume-chain and orchestration scripts for $\pi_0$ + SAILOR,” GitHub commit. Available: https://github.com/piroo8/Sailor_Pi0/commit/9bb1cb207ffda00c5555b16cb0d458d96c225ab9